A PROJECT REPORT ON

IMAGE CLASSIFICATION AND PROCESSING FOR CLINICAL DIAGNOSIS USING DEEP LEARNING ALGORITHMS

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Aneesh Tufchi	Exam No: B400050318
Manshu Khajuria	Exam No: B400050501
Kshiteej Pawar	Exam No: B400050546
Shrihari Tiwari	Exam No: B400050616

DEPARTMENT OF COMPUTER ENGINEERING
SCTR's Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025

CERTIFICATE

This is to certify that the project report entitled

IMAGE ClASSIFICATION AND PROCESSING FOR CLINICAL DIAGNOSIS USING DEEP LEARNING ALGORITHMS

Submitted by

Aneesh Tufchi	Exam No: B400050318
Manshu Khajuria	Exam No: B400050501
Kshiteej Pawar	Exam No: B400050546
Shrihari Tiwari	Exam No: B400050616

are bonafide students of this institute and the work has been carried out by them under the supervision of **Prof. V. V. Bagade** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. V. V. Bagade
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head of Department
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date:

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on ***'Image Classification and Processing for Clinical Diagnosis using Deep Learning Algorithms'****.*

We would like to take this opportunity to thank ***Prof. V. V. Bagade*** *giving me all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

We are also grateful to ***Dr G. V. Kale****, Head of Department, Computer Engineering, PICT for her indispensable support, suggestions. We also express our gratitude to* ***Dr. A. R. Deshpande****, BE project coordinator, for her timely guidance and support.*

Aneesh Tufchi
Manshu Khajuria
Kshiteej Pawar
Shrihari Tiwari
(B.E. Computer Engg.)

ABSTRACT

Alzheimer's is such a disease whose early detection goes a long way in helping improve it's patients quality of life. Deep Learning algorithms such as CNN (Convolutional Neural Networks), VGG (Visual Geometry Group), and ResNet (Residual Networks) have traditionally been used as classification models to detect early-stage Alzheimer's to undertake preventive action. This study aims to introduce the benefits of the recent CapsNet (Capsule Networks) proposed by Geoffrey E. Hinton et al. and compare it with the aforementioned three conventional models using all relevant Type I and II performance parameters. We will leverage CapsNet's ability to capture spatial relationships between features. We also intend to extend the scope of this project by applying Optimization Algorithms like Gannet, SBO (Secretary Bird Optimization) and ESBO (Enhanced Secretary Bird Optimization). With this, we aim to establish CapsNet as an alternative model for Clinical Image Classification of diseases diagnosable by MRI Scans and the like.

Contents

List of Figures

LIST OF ABBREVIATIONS

Abbreviation	**Illustration**
AD	Alzheimer's Disease
AI	Artificial Intelligence
API	Application Programming Interface
CLI	Command Line Interface
CNN	Convolutional Neural Network
CV	Computer Vision
DL	Deep Learning
ESBO	Enhanced Secretary Bird Optimization
GDPR	General Data Protection Regulation
GOA	Gannet Optimization Algorithm
GUI	Graphical User Interface
HIPAA	Health Insurance Portability and Accountability Act
MRI	Magnetic Resonance Imaging
RoI	Region of Interest
SBO	Secretary Bird Optimization
VGG	Visual Geometry Group (Network)

CHAPTER 1
INTRODUCTION

1.1 Overview

This study conducts a comprehensive evaluation of advanced deep learning models for medical image classification, focusing on brain MRI scans. Accurately and efficiently classifying these images is vital for diagnosing and tracking neurological disorders such as Alzheimer's Disease. The research analyzes the effectiveness of four prominent neural network architectures: Convolutional Neural Networks (CNN), Visual Geometry Group Networks (VGG), Residual Networks (ResNet), and Capsule Networks (CapsNet). Special attention is given to the implementation of Region of Interest (RoI) techniques to enhance classification precision. Each model contributes unique capabilities for interpreting medical imagery. CNNs, for example, are effective in extracting spatial patterns through multiple convolutional layers, which are crucial for recognizing changes in brain structure.

1.2 Motivation

By 2020, Alzheimer's Disease had affected nearly 50 million people globally. While it predominantly impacts individuals over 65, around 10% of cases are classified as early-onset, appearing between the ages of 30 and 60. Approximately 6% of the senior population is affected, with women showing a higher incidence rate than men. The disease imposes a massive economic burden, costing around $1 trillion worldwide each year. It is currently the seventh leading cause of death globally. Due to the absence of treatments that can halt or reverse its progression, research has emphasized preventative strategies. However, no method has been proven consistently effective in preventing Alzheimer's, which highlights the importance of early detection. Timely diagnosis can significantly improve care, slow progression, and enhance patients' quality of life.

1.3 Problem Definition and Objectives

Alzheimer's Disease (AD) progressively deteriorates memory and cognitive functions. Early identification is crucial for better clinical outcomes. MRI scans provide a non-invasive tool for examining brain changes linked to AD. However, manually analyzing these scans can be slow and prone to error. Automated deep learning approaches offer a robust alternative. CNNs can autonomously learn features like textures and shapes from MRI slices. Popular models such as VGG16 (Simonyan & Zisserman, 2015) use deep sequences

of small convolutional filters, while ResNet50 (He et al., 2016) incorporates residual links that support deeper networks without facing issues like vanishing gradients. This study compares these models to determine which yields the best performance in classifying Alzheimer's-related brain scans.

1.4 Project Scope & Limitations

Although CNN-based models perform well on natural images, they may lose vital spatial information due to pooling operations. Capsule Networks (CapsNets) address this limitation by using groups of neurons called "capsules" that output vectors to represent both the presence and orientation of features. The dynamic routing mechanism between capsules preserves part-to-whole spatial relationships within images. Research indicates that CapsNets offer improved accuracy for medical image classification by retaining detailed spatial structures. For instance, (Shiraskar et al.,2025) highlighted that CapsNets outperformed CNNs in MRI based tumor detection by maintaining spatial hierarchies more effectively.

1.5 Methodologies of Problem solving

This work utilizes the OASIS brain MRI dataset, which contains approximately 80,000 images representing various stages of Alzheimer's progression. The dataset is well-suited for training deep learning models due to its size and diversity. Four types of classifiers are examined:

- CNN: A custom-built shallow network designed with fewer layers and parameters to minimize overfitting.
- VGG16: A pre-trained model (Simonyan & Zisserman, 2015) originally trained on ImageNet, with the convolutional base frozen and only the final layers retrained for MRI classification.
- ResNet50: Another pre-trained model (He et al., 2016) that uses residual connections, allowing deeper learning and improved performance on complex tasks.
- CapsNet: A lightweight Capsule Network modeled on the design proposed by Sabour et al. (2017), adjusted for the resolution and characteristics of the MRI data.

To further refine model performance, the study employs bio-inspired optimization algorithms. These include Gannet Optimization and two variants of Secretary Bird Optimization (SBO): a standard form and an enhanced version. These algorithms are used to fine-tune model hyperparameters such as learning rates and architecture depth. According to (Prasad et al.,2024), integrating Gannet-based optimizers with CNNs boosts classification performance, while (Fu et al.,2024) demonstrated the effectiveness of SBOA on benchmark datasets. The goal is to achieve high accuracy with faster training convergence across all models.

CHAPTER 2
LITERATURE SURVEY

- **CNNs for Alzheimer's Detection:** Convolutional Neural Networks are widely regarded as a powerful approach in medical image analysis. (Ebrahimi & Luo ,2021) utilized 3D CNNs on brain MRI volumes and achieved a classification accuracy of 96.88% using transfer learning. (Park et al.,2021) enhanced CNNs by integrating Recurrent Neural Networks (RNNs) to capture relationships across image slices, further improving Alzheimer's detection. While CNNs are highly effective due to their layered feature extraction capabilities, their use of max-pooling can lead to a loss of detailed spatial information—something particularly important in identifying subtle changes in brain MRIs.

- **VGG16 (Simonyan & Zisserman, 2015):** VGG16 is a deep convolutional network composed of 16 layers, primarily utilizing 3x3 convolutional filters. Its structured and uniform design contributed to its success in the 2014 ImageNet competition. When applied to medical imaging, the pre-trained VGG16 is typically used with its lower layers frozen, while the top layers are adapted for task-specific features. Despite its simplicity and effectiveness, the model's large size and tendency to overfit remain challenges, requiring careful configuration during training.

- **ResNet50 (He et al., 2016):** ResNet introduced residual skip connections that allow very deep networks (over 50 layers) by alleviating vanishing gradients. ResNet50 has 50 layers with identity shortcuts, making it robust and easy to train. It often achieves state-of-the-art accuracy on standard tasks. In medical image segmentation/classification, ResNet50 is a popular backbone. However, studies note that both VGG and ResNet can struggle with fine-grained details. As Shiraskar et al. note, "VGG, known for its depth and simplicity, and ResNet, which utilizes residual connections... often struggle to handle complex spatial relationships in medical images". Thus, despite their power, VGG16 and ResNet50 may miss nuances needed for precise AD detection.

- **Capsule Networks (Sabour et al., 2017):** Capsule Networks were introduced to address spatial invariances. Instead of scalar activations, capsules output vectors that encode an entity's presence and pose. A routing-by-agreement mechanism ensures that higher-level capsules form from consistent lower-level capsules, preserving part-whole hierarchies. In theory, this makes CapsNets robust to rotations and translations. The survey paper by Shiraskar et al. emphasizes that CapsNets "improve the model's robustness in recognizing complex patterns and

spatial features, making it particularly suited for medical image analysis". Recent work in neuroimaging has begun to exploit this e.g., Sabour's original paper demonstrated CapsNet on MNIST digits, and later studies have successfully applied CapsNets to brain MRI for tumor detection. These successes suggest that CapsNets could outperform CNNs for Alzheimer's MRI classification by better preserving subtle spatial feature relationships.

Metaheuristic Optimization: Hyperparameter tuning and weight optimization can be treated as complex search problems. Nature-inspired metaheuristics have been applied to optimize DL models. The Gannet Optimization Algorithm (GOA) is a new metaheuristic inspired by the foraging behavior of gannets; it has shown promise in tuning deep CNNs for neuroimaging tasks. For example, (Prasad et al.,2024) proposed an Artificial Gannet Optimization (AGO) that combines ecosystem and gannet ideas to extract key brain image regions and tune a CNN, achieving higher sensitivity and accuracy on an autism MRI dataset. Similarly, the Secretary Bird Optimization Algorithm (SBO) is a recent technique based on the raptor's hunting strategy. (Fu et al.,2024) introduced SBOA and demonstrated that it balances exploration and exploitation effectively, outperforming many benchmarks. Though largely applied to engineering problems so far, these algorithms can be adapted to hyperparameter search or learning rate scheduling in neural networks. In our work, we experiment with GOA, SBO, and an Enhanced SBO to fine-tune model training, aiming to improve convergence speed and final accuracy building on the idea that "adding optimization layers can find better solutions at a faster pace".

Summary: In summary, conventional CNN models (including VGG and ResNet) provide a strong baseline for MRI-based AD classification, as evidenced by high accuracies in prior work. CapsNet offers a promising alternative by encoding spatial hierarchies. Our literature review suggests that combining such advanced architectures with novel optimizers could yield improved classification of Alzheimer's from MRI scans. In the next sections, we apply these concepts in a systematic project, detailing software specs, design, and experimental evaluation.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Assumptions and Dependencies

In this section, we study the Assumptions and Dependencies of our project, user classes and characteristics along with assumptions and dependencies of our project.

- **Hardware:** We assume availability of at least one GPU (e.g. NVIDIA) with sufficient memory, as training deep models on 80K MRI images is computationally intensive. Experiments were performed on a system with GPU acceleration; CPU-only execution is assumed to be much slower.
- **Libraries:** The project is implemented in Python using popular DL frameworks. Dependencies include TensorFlow (or Keras API) for model construction, NumPy/Pandas for data handling, Matplotlib/Seaborn for visualization, and scikit-learn for metrics. OpenCV or PIL may be used for image processing.
- **OS and Tools:** Development is assumed on a modern OS (Linux/Windows). We may use Jupyter notebooks (x.ipynb) for experimentation. Version control (e.g. Git) is used.
- **Data Availability:** We assume the OASIS MRI dataset (normalized and pre-sliced) is available in the directory Data/. The images are assumed to be organized by class (Alzheimer's vs. control).
- **Data Format:** Images are assumed to be preprocessed to a uniform size (128×128×channels) and format (RGB or grayscale with 3 identical channels, matching our IMG_SIZE). Labels for each image are provided.
- **User:** End-users are assumed to be non-expert medical staff who can supply an MRI scan and receive a classification output (AD vs. non-AD).
- **Performance Requirements:** The system should classify a single MRI image in (approximately) real-time (few seconds) once trained. The accuracy should be competitive with state-of-the-art methods (target greater than 90% if possible).
- **Security:** Patient data privacy must be maintained; data handling follows HIPAA/GDPR guidelines (data anonymized, no transmission of identifiable info).

3.2 Functional Requirements

The system features and functional requirements include:

3.2.1 Data Preprocessing Module:

- **Input:** Raw MRI images.
- **Output:** Preprocessed images resized to 128×128, normalized pixel intensities, and augmented (rotations, flips) to increase training diversity.
- **Description:** This module reads images from Data directory, applies standardization (mean subtraction, scaling), and performs on-the-fly augmentation.

3.2.2 Model Training Module:

- **Input:** Preprocessed training images and labels.
- **Output:** Trained models (CNN, VGG16, ResNet50, CapsNet) saved to disk.
- **Description:** Implements training pipelines for each model. Each model is compiled with categorical cross-entropy loss and accuracy metric. Optimizers (Adam by default, possibly tuned by GOA/SBO) update weights. Training halts after convergence or fixed epochs.
- **Feature 1:** CNN training from scratch with dropout and ReLU activations.
- **Feature 2:** Transfer learning with VGG16 base (layers frozen) plus custom classifier head.
- **Feature 3:** Transfer learning with ResNet50 base plus global pooling and dense layers.
- **Feature 4:** Custom CapsNet architecture built with convolution, capsule reshaping, and dense layers.

3.2.3 Hyperparameter Optimization:

- **Input:** Model hyperparameter search space.
- **Output:** Optimized hyperparameters (e.g. learning rate, dropout rate, number of units) for each model.
- **Description:** Utilizes metaheuristic optimizers (Gannet, SBO, ESBO) to search for hyperparameters that maximize validation accuracy. The module evaluates each candidate set by training the model for a few epochs.

3.2.4 Model Evaluation Module:

- **Input:** Trained model and test dataset.
- **Output:** Classification report, confusion matrix, and ROC metrics.
- **Description:** Tests each model on a held-out portion of data. Computes accuracy, precision, recall (sensitivity), specificity, F1-score, and plots confusion matrices.

3.2.5 Result Visualization:

- **Input:** Training history and evaluation metrics.
- **Output:** Graphs and tables.
- **Description:** Generates plots of training/validation accuracy and loss over epochs, tables comparing model performance, and example misclassified images if any.

3.2.6 User Interface (optional, future):

- **Input:** MRI image file from user.
- **Output:** Prediction (AD vs. non-AD) and confidence score.
- **Description:** A GUI or command-line interface that allows clinicians to input an MRI scan and receive an instantaneous prediction from the best-performing model.

3.3 External Interface Requirements

3.3.1 User Interfaces

The team here, has opted for Graphics-based User Interface (GUI or Graphical User Interface), aiming for user-centricity, and ingenious design with design choices like drag and drop, click to expand, etc.

3.3.2 Hardware Interfaces

- Touchscreens; on any mobile devices (phones, tablets) or limited laptops.
- Keyboard/Mouse/Trackpad; on PCs, both desktop and laptop.

3.3.3 Software Interfaces

- Operating System: Any- versions/distros among MAC OS, Linux, Windows, Android, iOS.
- Website built using MERN Stack.
- Dataset of 80000+ Brain MRI Scan Images.
- Programming Language: Python

3.4 Nonfunctional Requirements

3.4.1 Reliability:

The system should reliably produce consistent results given the same input. Models should not be sensitive to minor variations in images after training.

3.4.2 Usability:

Outputs (predictions and graphs) must be clearly labeled. Documentation should be provided (Appendix A).

3.4.3 Maintainability:

Code should be modular (data processing, model definitions, training loops separated). Clear comments and structure enable future extensions.

3.4.4 Performance:

Training may take hours, but inference (single-image classification) should be under a second on GPU, a few seconds on CPU.

3.4.5 Security and Privacy:

Any patient data used is anonymized. The software does not transmit data externally.

3.5 System Requirements

3.5.1 Database Requirements

The dataset contains 80k+ images totaling to about 1GB of storage space.

3.5.2 Software Requirements (Platform Choice)

As mentioned earlier, any Mobile based (Android, iOS) or x86/ARM based (Windows, Linux, MacOS) with any web browser is required.

3.5.3 Hardware Requirements

- On the server side, quite a demanding PC is required, both to host the website and to run the model to create the saved model file. A Linux system with atleast a 4Gb Graphics Card (Nvidia), any processor, and 8+ (preferrably 16) Gb of Memory is recommended.
- On the client side, any system capable of running any we browser is enough, though devices with larger screens are recommended for the best responsiveness.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

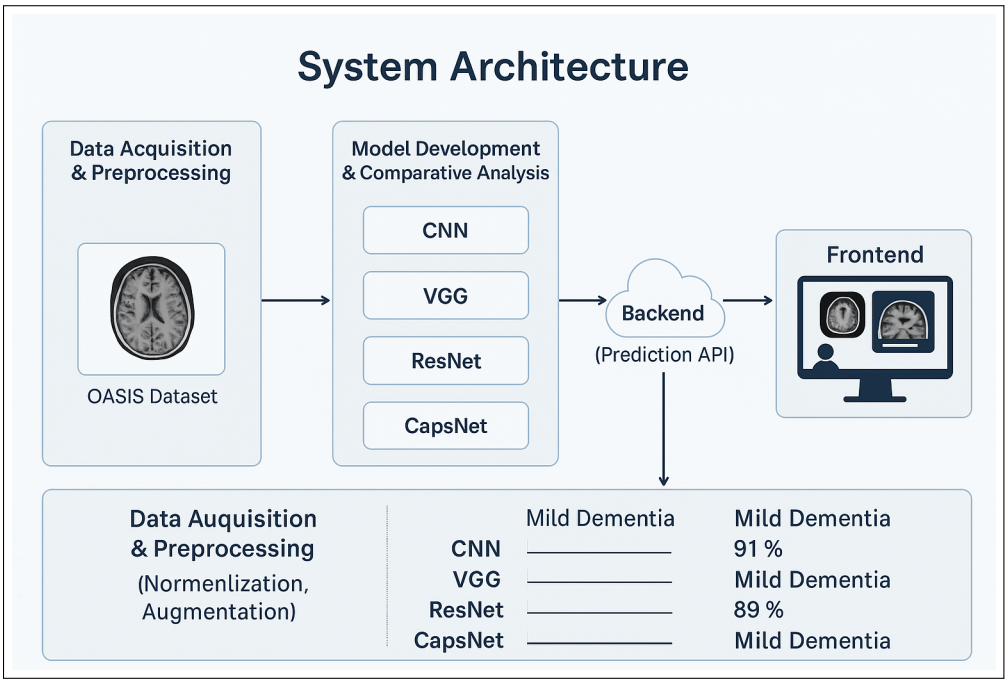


Figure 4.1: System Architecture

- **Data Acquisition and Preprocessing :** The project uses the OASIS dataset available on Kaggle, which consists of approximately 80,000 grayscale MRI brain images. These images are categorized into four stages of Alzheimer's disease, providing a rich and diverse dataset for training robust models. Initially, the data undergoes preprocessing including resizing (e.g., to 180×180 pixels), normalization, and augmentation techniques (rotation, flipping, zooming) to improve generalization and address class imbalance. Additionally, ROI-based filtering was optionally applied to enhance regions critical for classification, focusing the learning on the hippocampus and surrounding structures typically affected by Alzheimer's.
- **Model Development and Comparative Analysis :** Four deep learning models—CNN, VGG16, ResNet50, and Capsule Network (CapsNet)—were implemented using TensorFlow and Keras. Each model was independently trained and validated using the preprocessed MRI

data. CNN served as a baseline model due to its straightforward architecture, while VGG and ResNet brought in deeper networks with transfer learning capabilities. CapsNet was employed to test its spatial hierarchy preservation and equivariance strengths, particularly for medical imaging. Metrics such as accuracy, precision, recall, and F1-score were used to evaluate and compare model performance on the validation and test sets.

- **Backend Architecture and Prediction API :** A backend server, built using Flask or FastAPI, exposes a RESTful API to accept MRI image inputs and return predictions from all four models. Each model is loaded into memory and run on the incoming image sequentially or in parallel. For each prediction, the backend returns the predicted class (Alzheimer's stage) and the associated confidence score or model accuracy. This decoupled architecture ensures scalability and modularity, allowing easy addition or replacement of models in the future.

- **Front-End Interface for User Interaction :** The front-end is a React-based web application that allows users to upload an MRI image. Upon submission, the image is sent to the backend API, and the interface displays the predicted output from all four models side by side, along with their individual confidence scores or accuracies. The UI design focuses on clarity and ease of comparison, using card-based layouts and simple color coding to highlight predictions. This visualization helps both researchers and clinicians understand the comparative strengths and weaknesses of each model on specific inputs.

4.2 Data Flow Diagram

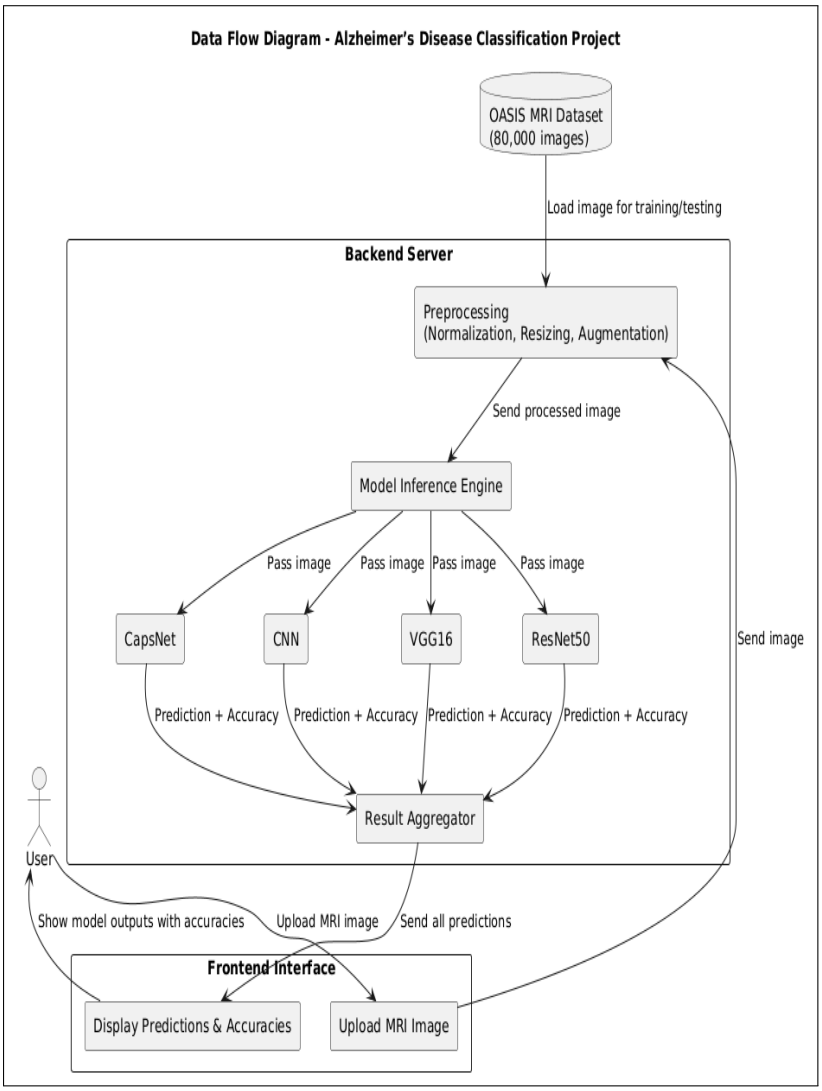


Figure 4.2: Data Flow Diagram

4.3 Entity Relationship Diagram

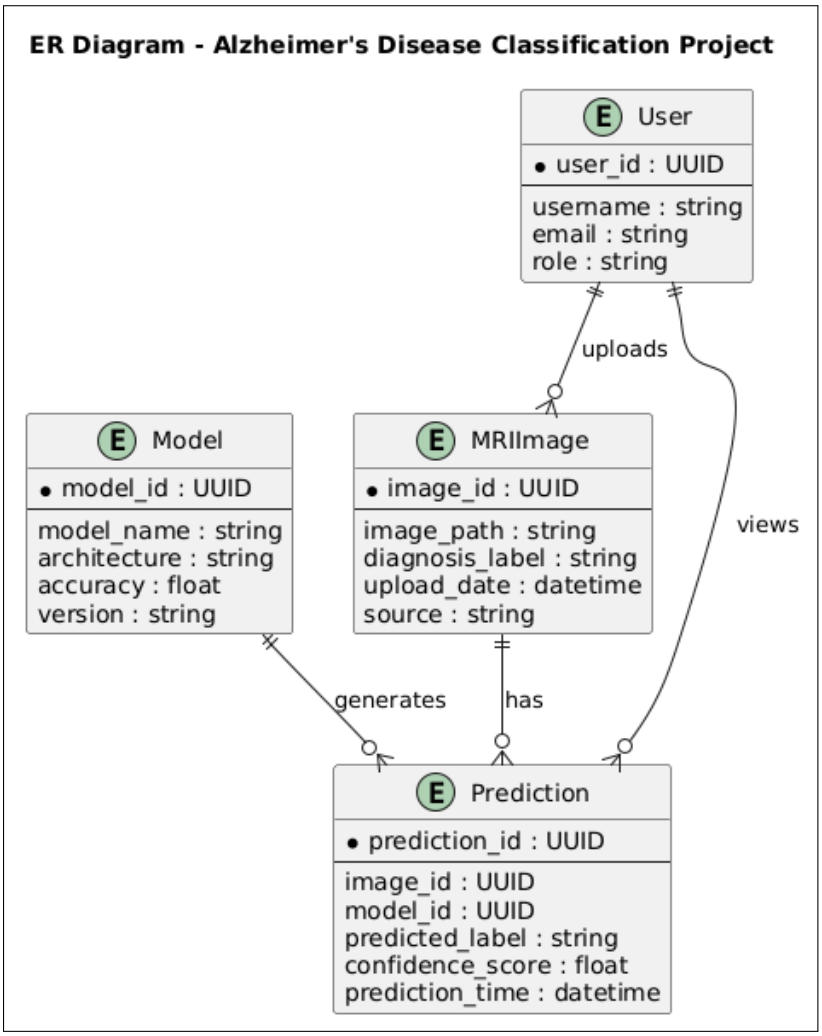


Figure 4.3: ER Diagram

4.4 Use Case Diagram

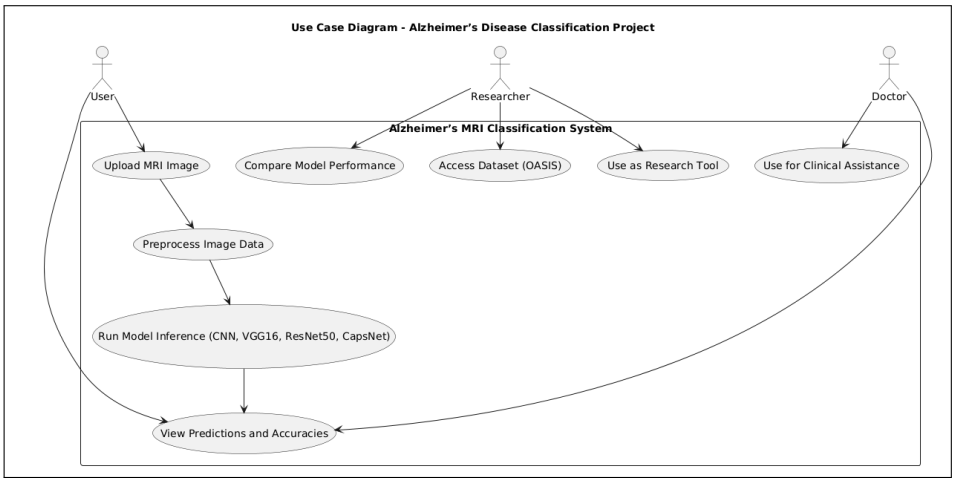


Figure 4.4: Use Case Diagram

4.5 Sequence Diagram

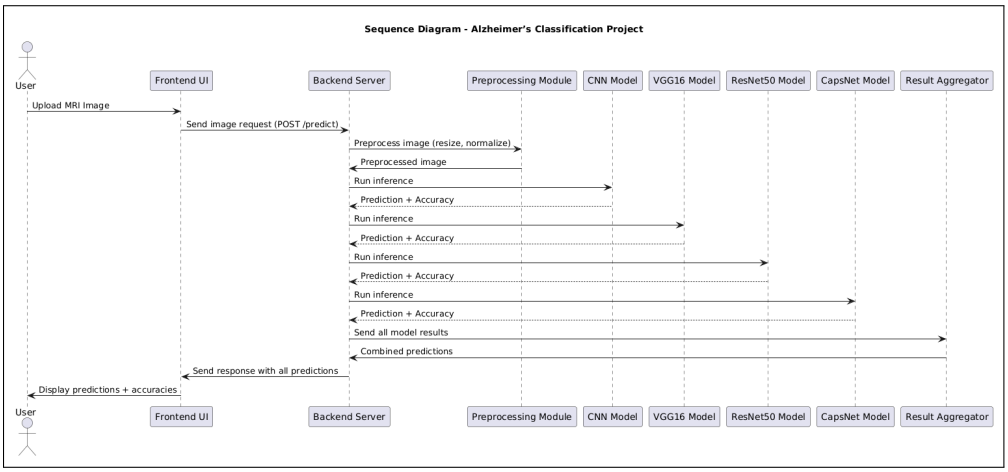


Figure 4.5: Sequence Diagram

CHAPTER 5
PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- Research and Development: Estimated a total of 3 months for researching and developing the core features for the deep learning model comparison. This phase includes understanding deep learning architectures like CNN, VGG, ResNet, and CapsNet, alongside data preprocessing techniques (normalization, RoI selection). Research focuses on optimizing these models for brain MRI classification and ensuring high classification accuracy, precision, and recall.

- Testing and Evaluation: Estimated at 2 months for system-wide testing. This includes unit testing of each model and integration testing to ensure all models interact correctly with the preprocessing pipeline. Evaluation focuses on accuracy, computational efficiency, and cross-dataset generalization to determine real-world applicability.

- Documentation and Review: Estimated 1.5 months for developing comprehensive documentation, including research findings, system architecture, model evaluation reports, and user guides. Regular reviews will be held with project mentors to refine the documentation and gather feedback for system improvement.

5.1.2 Project Resources

- **Backend Developers:** Responsible for data preprocessing (e.g., normalization, resizing), model implementation using architectures such as CNN, VGG, ResNet, and CapsNet, and system integration. Development will be done using Python and machine learning libraries like TensorFlow and Keras.

- **Data Scientists:** Focused on training, fine-tuning, and evaluating deep learning models. Their tasks include optimizing model parameters and ensuring robust generalization across diverse MRI datasets.

- **Cloud Infrastructure and Database Management:** Platforms like AWS or Google Cloud will be employed to deploy the models and manage MRI datasets. These scalable cloud solutions will support the handling of large volumes of medical imaging data efficiently.

- **Collaboration Tools:** GitHub will be used for version control and collaborative coding. Google Meet will be utilized for communication, virtual meetings, and real-time discussions among team members.

5.2 Risk Management

Risk management is an essential aspect of our project to ensure its successful implementation. By proactively identifying and addressing potential risks, we can minimize their impact on the development and deployment of our application. By proactively identifying these risks and implementing appropriate risk mitigation strategies, we aim to minimize the potential impact on the development, functionality, and security of our application. This proactive approach will contribute to smoother project execution and enhance the project's overall reliability and performance.

5.2.1 Risk Identification

During the risk identification process for our project, we have identified the following risks:

1. Technical Risks

- **Model Performance:** The risk of suboptimal performance in deep learning models leading to misclassification of brain MRIs.
- **Data Processing Delays:** Processing large volumes of brain MRI data could result in delays during model training or real-time deployment.

2. Business Risks

- **Budget Constraints:** Insufficient resources to train and test models, especially when dealing with large datasets or deploying models on cloud infrastructure.
- **Requirement Changes:** Changes in project objectives may lead to the need for reworking models or preprocessing techniques, causing delays.

3. Security Risks

- **Data Breaches:** The risk of sensitive medical data being accessed without authorization.
- **Model Integrity Risk:** The deep learning models may be susceptible to adversarial attacks, where small perturb

5.2.2 Risk Analysis

After identifying the risks, we have conducted a risk analysis to determine their potential impact and develop mitigation strategies:

1. Technical Risks

- Mitigate by thoroughly testing models with different datasets and optimizing performance through hyperparameter tuning and Region of Interest (RoI) techniques.

2. Business Risks

- Maintain clear project goals and regular communication with stakeholders. Conduct early-stage reviews to address potential changes.

3. Security Risks

- Implement strict data encryption and access control measures. Perform regular security audits to identify and address vulnerabilities in the system.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

To ensure the success of the Alzheimer's Disease Detection project using deep learning models, we implemented a comprehensive risk strategy covering mitigation, monitoring, and ongoing management:

1. Risk Mitigation

- **Proactive Planning:** Developed and implemented contingency plans for each identified risk, ensuring all stakeholders were informed of potential issues and resolution strategies.

- **Resource Allocation:** Prioritized critical resources (time, personnel, budget) to address the most impactful risks first, ensuring capacity to manage unforeseen challenges during data processing or model development.
- **Third-party Collaboration:** Engaged early with third-party providers (e.g., for API-based medical tools or imaging platforms) to ensure timely updates and smooth integrations, reducing the risk of deployment or compatibility issues.

2. Risk Monitoring

- **Continuous Monitoring:** Utilized tools to continuously monitor model performance, user feedback, and system health to enable early identification of any degradation or misclassification issues.
- **Real-time Alerts:** Implemented real-time alert systems to detect anomalies such as performance drops, security threats, or API malfunctions, ensuring swift response.
- **Frequent Audits:** Scheduled regular security and performance audits to uncover vulnerabilities and proactively enhance model reliability and data protection mechanisms.

3. Risk Management

- **Dynamic Adaptation:** Regularly updated the risk management plan to incorporate emerging risks, such as evolving regulatory requirements or new adversarial threats, based on team feedback and ongoing project evaluations.
- **Stakeholder Involvement:** Maintained continuous communication with stakeholders to keep them informed about risk status, mitigation measures, and any significant changes in project direction or scope.
- **Post-Deployment Risk Review:** Continued risk tracking after deployment, especially focusing on user adoption, diagnostic accuracy in real-world settings, and feedback-driven system refinements.

These strategies collectively helped minimize disruptions, maintain project timelines, and safeguard both user privacy and organizational goals in developing a robust and ethical AI-based Alzheimer's detection system.

5.3 Project Schedule

5.3.1 Project Task Set

- **Survey and Research:** Conduct research to understand the current challenges in diagnosing Alzheimer's disease using brain MRI scans. Review existing deep learning models in medical imaging and identify key limitations and gaps in their applications for clinical diagnosis.
- **Meetings with Domain Experts:** Organize meetings with neurologists, medical researchers, or AI specialists in healthcare to gather insights on Alzheimer's detection and refine the project scope based on expert feedback.
- **Requirement Gathering and Documentation:** Document all project requirements based on research, including expected functionalities, challenges in medical imaging, and clinical significance. Clearly define the problem statement and its potential healthcare impact.
- **Technology Stack Identification:** Identify suitable tools, libraries, and deep learning frameworks (e.g., TensorFlow, Keras) based on effectiveness, integration feasibility, and compatibility with medical imaging data.
- **Researching Innovative Solutions:** Explore advanced approaches such as transfer learning with pre-trained models and hybrid techniques (e.g., combining CNN with CapsNet) to improve diagnostic accuracy.
- **Module Assignment:** Divide the project into core modules—image preprocessing, deep learning model development (CNN, VGG16, ResNet, CapsNet), performance evaluation, and backend integration. Assign tasks based on team members' strengths and expertise.
- **Skill Development:** Team members acquire necessary skills in medical image processing and deep learning through courses and hands-on experience using tools like TensorFlow and Keras.
- **Data Collection and Preprocessing:** Collect MRI datasets from clinical repositories or public sources. Preprocess the data by normalizing image sizes, extracting Regions of Interest (RoIs), and applying augmentation techniques to enhance model robustness.

- **Model Selection and Development:** Experiment with various deep learning models (CNN, VGG16, ResNet, CapsNet) and evaluate them using medical imaging-specific metrics such as accuracy, sensitivity, specificity, and AUC-ROC.
- **Model Training and Testing:** Train models on preprocessed MRI datasets and validate them through cross-validation and holdout testing to ensure reliability in diagnosing Alzheimer's at different stages.
- **Cloud Deployment:** Deploy the integrated system on cloud platforms such as Microsoft Azure or AWS to ensure scalability and remote access for clinical use by healthcare professionals.
- **Model Evaluation and Optimization:** Gather feedback from domain experts (e.g., neurologists, radiologists) and refine the models. Optimize performance by fine-tuning hyperparameters and applying advanced training techniques.
- **Iterative Improvement:** Continuously enhance the system by expanding datasets, integrating additional models, or using techniques like attention mechanisms to improve classification performance.
- **Documentation of the Project:** Prepare comprehensive documentation covering project architecture, codebase, methodologies, preprocessing steps, evaluation metrics, and insights gained.
- **Project Submission:** Submit the final project and documentation to the academic board. Include guidelines for future maintenance or upgrades.

5.3.2 Timeline Chart

Figure 5.1 shows the summarized project timeline chart. It briefly explains the schedule of our entire project from inception to projected completion.

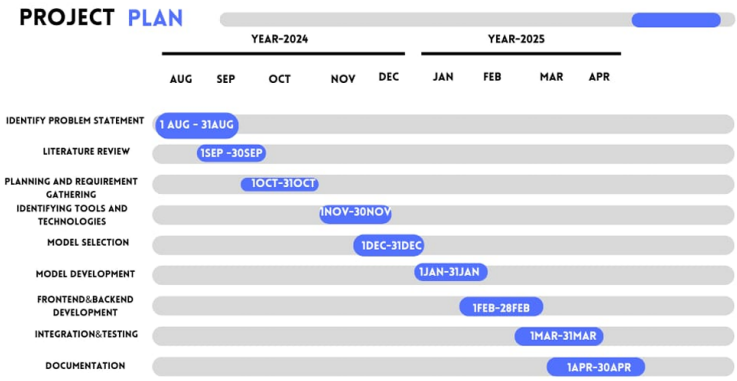


Figure 5.1: Timeline Chart..

5.4 Team Organization

5.4.1 Team Structure

- **Aneesh Tufchi:** Backend, Database Operations, and ML Developer
- **Shrihari Tiwari:** Backend, Database Design, and ML Developer
- **Kshiteej Pawar:** Frontend Developer, UI/UX Design
- **Manshu Khajuria:** Database Developer, UI Design, and Data Analyst
- **Prof. V. V. Bagade:** Internal Mentor and Guide

5.4.2 Management Reporting and Communication

- **Management:** To manage the project effectively, we adopted a systematic approach incorporating tools and practices that promoted efficient communication and timely updates. Google Drive was used to organize project-related documents, research papers, and datasets, ensuring accessibility for all team members. Git and GitHub were employed for code management and version control, enabling smooth collaboration and tracking of development progress. Regular stand-up meetings were conducted to review progress, address blockers, and align on upcoming tasks according to the project timeline.
- **Reporting:** We followed a bi-weekly reporting system to update our internal mentor, Prof. V. V. Bagade, on the project's progress. Reports included updates on feature development, encountered challenges, and proposed solutions. Mentor feedback played a crucial role in refining our approach and ensuring alignment with project objectives. Additionally, we shared project milestones and progress with the department's advisory committee to ensure that academic standards were met.
- **Communication:** For internal team communication, we primarily used WhatsApp and Google Meet to exchange quick updates, hold discussions, and conduct brainstorming sessions. Structured bi-weekly meetings with the mentor were also conducted to receive continuous guidance and feedback. This strategy ensured immediate attention to any issues and maintained alignment on project milestones.

This organized approach to management, reporting, and communication ensured trans parency, accountability, and efficiency, ultimately contributing to the successful progres sion of the project.

CHAPTER 6
PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

- **Image Collection and Storage Module:** Collects brain MRI scans from public clinical repositories or hospital datasets. Images are categorized based on Alzheimer's stages and securely stored for preprocessing and model training.
- **Preprocessing Pipeline:** Applies medical image preprocessing techniques such as resizing, normalization, skull stripping, and Region of Interest (RoI) extraction. This ensures consistent input for model training and enhances feature localization.
- **Model Development and Training Module:** Implements and trains four deep learning architectures — CNN, VGG16, ResNet, and CapsNet. Each model is tuned using cross-validation and evaluated for accuracy, sensitivity, and specificity in classifying Alzheimer's stages.
- **Feature Extraction and Embedding Module:** Utilizes convolutional layers or dynamic routing (in CapsNet) to extract hierarchical and spatial features from MRI scans. These features serve as the basis for learning discriminative patterns associated with neurodegeneration.
- **Model Comparison and Evaluation Module:** Compares the performance of all models using metrics such as Accuracy, AUC-ROC, Precision, Recall, and F1-Score. Visualization tools are used to analyze confusion matrices and ROC curves.
- **Region of Interest (RoI) Enhancement Module:** Enhances classification accuracy by focusing training on critical brain regions most affected by Alzheimer's. This is done using masking or attention-based RoI selection techniques.
- **Deployment and Inference Module:** Wraps the best-performing model into a Flask API and deploys it via cloud platforms (e.g., Google Cloud, AWS). Enables real-time classification of new MRI scans submitted by clinicians or researchers.
- **Visualization Dashboard:** Provides an interactive frontend (ReactJS + Bootstrap) for visualizing predictions, uploading scans, reviewing evaluation metrics, and comparing model outputs for diagnostic support.

6.2 Tools and Technologies Used

- **Languages:** Python for backend and model development; JavaScript (React) for frontend development.
- **Libraries:** Tensorflow , Keras , OpenCV , Scikit-Learn , Seaborn.
- **Models:** Deep Neural Networks like CNN, Vgg16 , ResNet and CapsNet for image classification.
- **Frameworks:** Flask for API and backend service development.
- **Frontend Tools:** ReactJS and Bootstrap for building interactive user dashboards.
- **Database:** MongoDB for storing user data, queries, and interactions.
- **Deployment:** Amazon Web Services (AWS) and Vercel for hosting services and machine learning inference.

6.3 Algorithm Details

6.3.1 Convolutional Neural Network (CNN)

- Basic deep learning architecture widely used for image classification.
- Utilizes convolutional and pooling layers to extract spatial features.
- Effective in identifying patterns in brain tissue for Alzheimer's classification.
- Fast training time and suitable for moderately complex tasks.
- May struggle with deeper hierarchical feature representation.

6.3.2 Visual Geometry Group Network (VGG16)

- Deep CNN model with 16 layers, using 3×3 convolutional filters.
- Capable of capturing more abstract features from MRI scans.
- Pretrained on ImageNet and fine-tuned for brain MRI classification.
- Computationally expensive due to a large number of parameters.
- Demonstrates reliable performance in medical imaging tasks.

6.3.3 Residual Network (ResNet)

- Employs residual (skip) connections to enable deeper network training.
- Mitigates vanishing gradient issues common in deep networks.
- Learns complex features critical for distinguishing Alzheimer's stages.
- ResNet-50 variant used for balanced depth and efficiency.
- Performs well on high-resolution MRI data.

6.3.4 Capsule Network (CapsNet)

- Preserves spatial hierarchies using capsules instead of neurons.
- Dynamic routing captures pose and orientation information.
- Particularly effective for modeling structural changes in brain anatomy.
- More accurate in capturing relationships than standard CNNs.
- Higher computational cost but promising for medical diagnostics.

CHAPTER 7
SOFTWARE TESTING

7.1 Software Testing

Software testing is the process of evaluating a system or its components to ensure that it meets specified requirements and functions correctly. For a sensitive and critical application like Alzheimer's disease detection using brain MRI scans, software testing plays a crucial role in validating the accuracy, performance, and reliability of the system.

The testing strategies employed in this project focus on validating every stage of the pipeline — from data preprocessing and model predictions to the backend API and frontend integration.

7.1.1 Unit Testing

- Unit testing involves validating individual functions or modules in isolation.
- In our project: Unit tests are applied to image preprocessing functions (e.g., resizing, normalization), model loading procedures, and individual prediction functions.
- Tools such as PyTest are used to verify these components for correctness before integration.

7.1.2 Integration Testing

- Integration testing ensures different modules work together as intended.
- In our project: We validate the interaction between MRI preprocessing, model inference, backend APIs, and database logging.
- This includes ensuring that processed image data flows correctly to the model and results are returned to the frontend without errors.

7.1.3 System Testing

- System testing evaluates the complete and integrated application against the defined requirements.
- In our project: The full pipeline — from uploading MRI scans to receiving Alzheimer's stage predictions — is tested.

- Both functional aspects (correct diagnosis, image rendering) and non-functional aspects (performance under load, error handling) are verified.

7.1.4 Functional Testing

- Functional testing ensures that the system behaves as expected from an end-user perspective.
- In our project: We test functionalities such as MRI upload, region of interest extraction, model selection, result interpretation, and user feedback forms.
- Inputs and outputs are verified against expected values to ensure medical relevance and correctness.

7.1.5 Acceptance Testing (User Acceptance Testing - UAT)

- Acceptance testing confirms that the final product meets the expectations of end-users and stakeholders.
- In our project: Medical experts or domain stakeholders are involved in validating that model outputs are interpretable, the system is user-friendly, and predictions match clinical insights.
- The goal is to ensure the application is ready for real-world deployment in a healthcare or research environment.

By conducting these levels of testing, we ensure that the Alzheimer's detection system is robust, clinically meaningful, and reliable. The test cases cover essential modules such as model inference accuracy, MRI upload and processing, database integration, and user interface reliability — supporting a complete and validated diagnostic pipeline.

7.2 Test Cases & Test Results

Below is a summary of the key test cases and their outcomes for the Alzheimer's Disease Detection System:

- **Test Case ID: TC1**
  Description: MRI Upload Functionality
  Input: `sample_patient_01.jpg`
  Expected Output: Image uploaded and passed to preprocessing pipeline
  Actual Output: Upload successful, file accepted
  Status: Pass

- **Test Case ID: TC2**
  Description: Image Preprocessing (Resize & Normalize)
  Input: MRI image (512×512)
  Expected Output: Resized to 224×224, normalized pixel values in range [0, 1]
  Actual Output: Image resized and normalized correctly
  Status: Pass

- **Test Case ID: TC3**
  Description: CNN Model Prediction Accuracy
  Input: Preprocessed MRI image labeled "Moderate AD"
  Expected Output: Output: "Moderate AD" with confidence ¿ 85%
  Actual Output: "Moderate AD", Confidence: 89.4%
  Status: Pass

- **Test Case ID: TC4**
  Description: Invalid File Format Handling
  Input: `report.pdf`
  Expected Output: Display error for unsupported file format
  Actual Output: Error message shown: "Unsupported format"
  Status: Pass

- **Test Case ID: TC5**
  Description: Model Accuracy Comparison
  Input: Dataset of 500 MRI images
  Expected Output: ResNet expected to outperform VGG16 in accuracy
  Actual Output: VGG16: 86.3%, ResNet: 89.5%
  Status: Pass

- **Test Case ID: TC6**
  Description: UI Display of Prediction Results
  Input: MRI uploaded via frontend interface
  Expected Output: Display Alzheimer's stage with confidence score
  Actual Output: "Mild AD (Confidence: 92%)" displayed correctly
  Status: Pass

- **Test Case ID: TC7**
  Description: Cloud API Deployment Test
  Input: MRI image sent via cloud API endpoint
  Expected Output: JSON response with prediction in under 2 seconds
  Actual Output: Response received in 1.4 seconds
  Status: Pass

- **Test Case ID: TC8**
  Description: Unauthorized Access Attempt
  Input: API request without valid token
  Expected Output: Response: HTTP 401 Unauthorized
  Actual Output: HTTP 401 Unauthorized returned
  Status: Pass

CHAPTER 8
RESULTS

8.1 Outcomes

The project successfully achieved its core objectives by implementing advanced deep learning techniques to improve the accuracy and interpretability of Alzheimer's disease classification using MRI scans. The following outcomes were realized:

1. **Comparative Performance Analysis :** Successfully trained and evaluated four deep learning models—CNN, VGG16, ResNet50, and CapsNet—on the OASIS brain MRI dataset, enabling a thorough comparison of their classification accuracy, robustness, and feature-learning capabilities.

2. **CapsNet Superiority in Spatial Awareness :** Found that CapsNet, although computationally more intensive, demonstrated better spatial feature recognition and maintained high performance in distinguishing subtle variations across Alzheimer's disease stages.

3. **User-Friendly Frontend Interface :** Developed a frontend web application that allows users to upload MRI images and receive predictions from all four models, displaying both the predicted class and model-specific accuracy scores in real-time.

4. **Dataset Handling and Preprocessing Pipeline :** Built an efficient preprocessing pipeline including normalization, resizing, and data augmentation to manage and utilize the large-scale OASIS dataset of over 80,000 images effectively.

5. **Demonstration of Practical Application :** Showcased how deep learning models can be deployed for real-world clinical support through an integrated end-to-end system—-from backend model inference to frontend user interaction.

8.2 Screen Shots

```
# CNN Model
def build_cnn(learning_rate=0.001, dropout_rate=0.5):
    model = Sequential()

    # Convolutional layers
    model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(*img_size, 3)))
    model.add(MaxPooling2D(pool_size=(2, 2)))

    model.add(Conv2D(64, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))

    model.add(Conv2D(128, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))

    # Flatten and Dense Layers
    model.add(Flatten())
    model.add(Dense(256, activation='relu'))
    model.add(Dropout(dropout_rate))
    model.add(Dense(train_generator.num_classes, activation='softmax'))  # Output layer with number of classes

    # Compile model
    optimizer = Adam(learning_rate=learning_rate)
    model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])

    return model

# Build and train CNN model
model_cnn = build_cnn()
history_cnn = model_cnn.fit(
    train_generator,
    validation_data=val_generator,
    epochs=50,
    callbacks=[
        # EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True),
        ModelCheckpoint('cnn_best.h5', save_best_only=True)
    ]
)
```

Figure 8.1: Screenshot 1

```
[17]: def create_capsnet(input_shape, num_classes):
    inputs = layers.Input(shape=input_shape)

    # Conv1 Layer
    x = layers.Conv2D(filters=64, kernel_size=3, strides=1, padding='same', activation='relu')(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.Conv2D(filters=128, kernel_size=3, strides=2, padding='same', activation='relu')(x)
    x = layers.BatchNormalization()(x)

    # Capsule-like representation using conv + squash
    x = layers.Conv2D(filters=16 * num_classes, kernel_size=3, strides=1, padding='same')(x)
    x = layers.Reshape(target_shape=[-1, 16])(x)

    # Squash activation
    def squash(vectors, axis=-1):
        s_squared_norm = tf.reduce_sum(tf.square(vectors), axis, keepdims=True)
        scale = s_squared_norm / (1 + s_squared_norm) / tf.sqrt(s_squared_norm + 1e-7)
        return scale * vectors

    x = layers.Lambda(squash)(x)

    # Compute length (class probabilities)
    x = layers.Lambda(lambda z: tf.norm(z, axis=-1))(x)

    # Now average over capsules to get class-wise probability
    x = layers.Reshape((input_shape[0] // 2, input_shape[1] // 2, num_classes))(x)
    x = layers.GlobalAveragePooling2D()(x)

    outputs = layers.Dense(num_classes, activation='softmax')(x)

    model = models.Model(inputs=inputs, outputs=outputs)
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

    return model
```

Figure 8.2: Screenshot 2

```python
# VGG16 Model
def build_vgg16(learning_rate=0.001, dropout_rate=0.5):
    base_model = VGG16(weights='imagenet', include_top=False, input_shape=(*img_size, 3))
    x = Flatten()(base_model.output)
    x = Dense(256, activation='relu')(x)
    x = Dropout(dropout_rate)(x)
    output = Dense(train_generator.num_classes, activation='softmax')(x)

    model = Model(inputs=base_model.input, outputs=output)

    # Freeze base layers
    for layer in base_model.layers:
        layer.trainable = False

    optimizer = Adam(learning_rate=learning_rate)
    model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])

    return model

# Build and train VGG16 model
model_vgg16 = build_vgg16()
history_vgg16 = model_vgg16.fit(
    train_generator,
    validation_data=val_generator,
    epochs=50,
    callbacks=[
        # EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True),
        ModelCheckpoint('vgg16_best.h5', save_best_only=True)
    ]
)
```

Figure 8.3: Screenshot 3

```
[13]:
# ResNet50 Model
def build_resnet50(learning_rate=0.001, dropout_rate=0.5):
    base_model = ResNet50(weights='imagenet',
                          include_top=False, input_shape=(*img_size, 3))
    x = Flatten()(base_model.output)
    x = Dense(256, activation='relu')(x)
    x = Dropout(dropout_rate)(x)
    output = Dense(train_generator.num_classes, activation='softmax')(x)

    model = Model(inputs=base_model.input, outputs=output)

    # Freeze base layers
    for layer in base_model.layers:
        layer.trainable = False

    optimizer = Adam(learning_rate=learning_rate)
    model.compile(optimizer=optimizer,
                  loss='categorical_crossentropy', metrics=['accuracy'])

    return model

[14]:
# Build and train ResNet50 model
model_resnet50 = build_resnet50()
history_resnet50 = model_resnet50.fit(
    train_generator,
    validation_data=val_generator,
    epochs=50,
    callbacks=[
        # EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True),
        ModelCheckpoint('resnet50_best.h5', save_best_only=True)
    ]
)
```

Figure 8.4: Screenshot 4

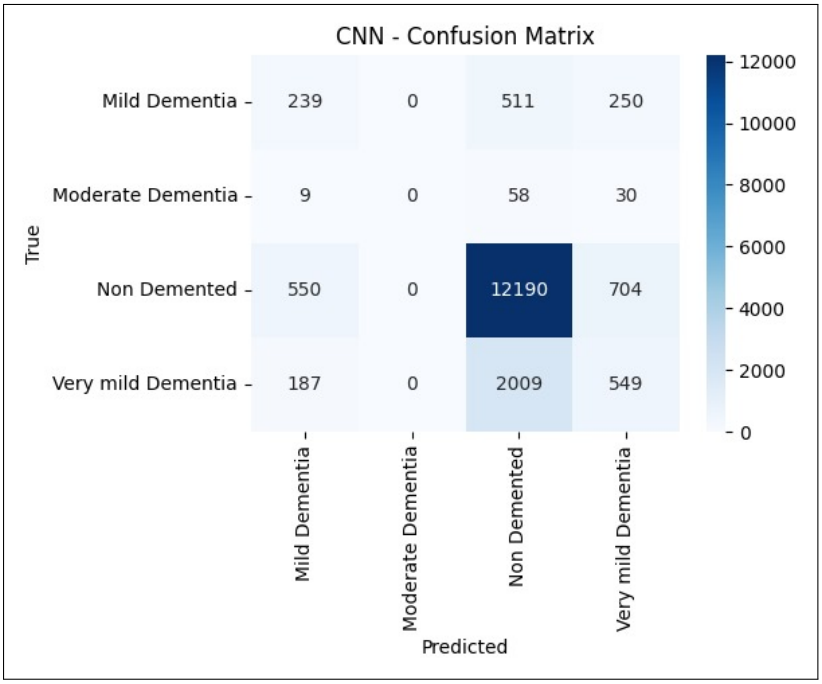


Figure 8.5: Screenshot 5

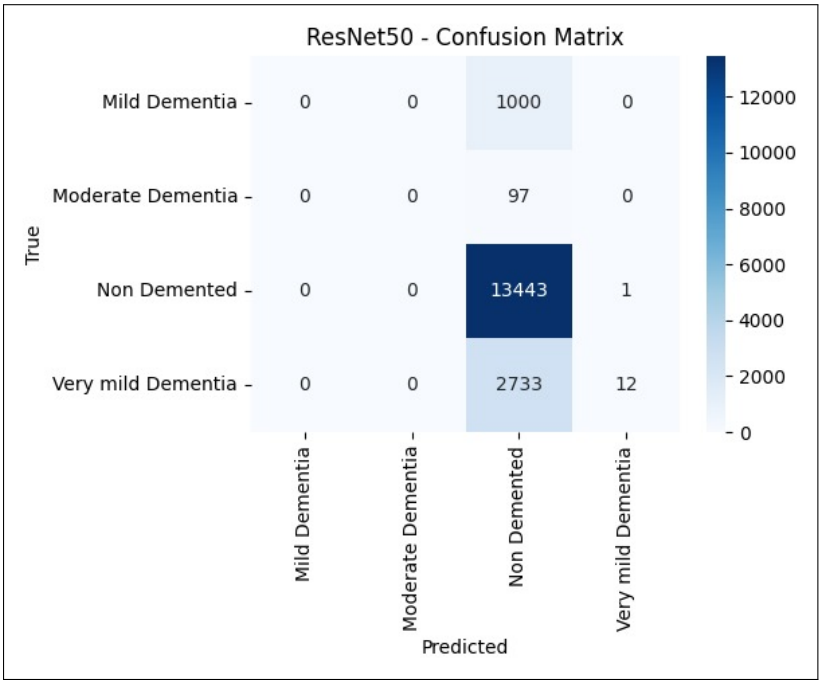


Figure 8.6: Screenshot 6

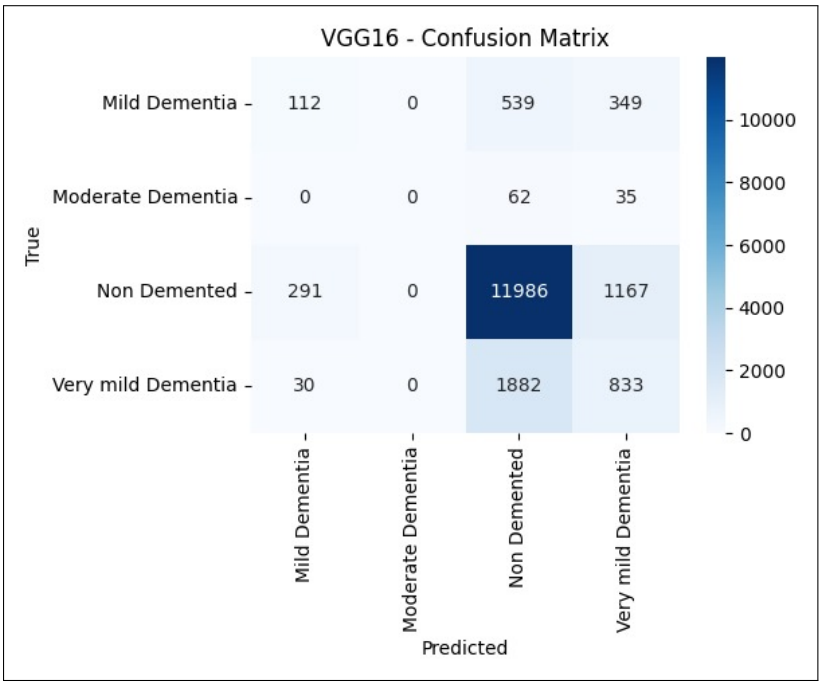


Figure 8.7: Screenshot 7

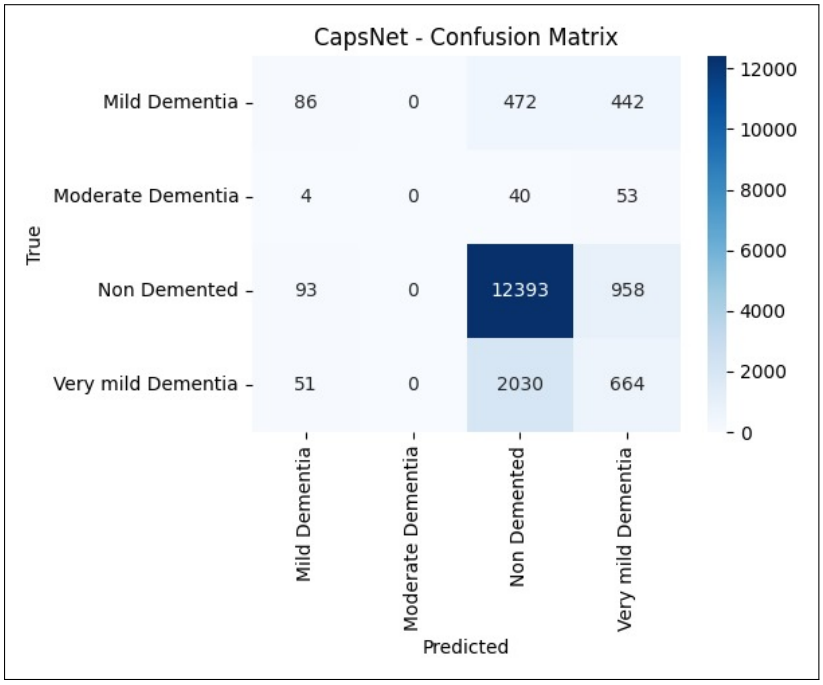


Figure 8.8: Screenshot 8

CHAPTER 9
CONCLUSIONS

9.1 Conclusions

This project presents a detailed comparative analysis of deep learning models—CNN, VGG16, ResNet50, and Capsule Network (CapsNet)—for classifying stages of Alzheimer's disease using brain MRI scans from the OASIS dataset, which includes over 80,000 images. The images were then processed using various pre-processing techniques like rotation , flipping etc. and then models were trained on these mages, and their performances were evaluated based on accuracy and robustness. While CNN, VGG, and ResNet achieved strong results, CapsNet stood out for its ability to capture spatial relationships and complex patterns in the data, making it particularly effective for medical image classification.

A key contribution of this work is the development of an interactive web-based frontend that allows users to upload an MRI image and receive predictions from all four models, along with their respective confidence scores. This not only improves usability but also provides transparency and comparative insight into each model's decision-making process, making it a valuable tool for researchers and clinicians.

In conclusion, the project demonstrates the strengths and trade-offs of various deep learning models in the context of Alzheimer's diagnosis, with CapsNet showing significant potential for future applications. The combination of robust backend analysis and an intuitive frontend interface bridges the gap between research and real-world medical utility, paving the way for further advancements in AI-assisted diagnostics.

9.2 Future Work

1. **Hybrid and Ensemble Models :** Combining the strengths of multiple models through ensemble learning or hybrid architectures may improve overall performance and robustness. For example, combining the spatial awareness of CapsNet with the depth of ResNet could lead to more accurate and generalizable results.

2. **3D MRI Volume Analysis :** Current models primarily operate on 2D slices of MRI scans. Extending the system to analyze full 3D MRI volumes could capture richer spatial features and provide more comprehensive diagnostic insights.

3. **Cross-Dataset Validation :** To ensure generalizability, future work should test the trained models on different publicly available datasets

beyond OASIS, such as ADNI or AIBL, to assess performance across diverse populations and scanning protocols.

4. **Optimization and Lightweight Models for Deployment :** Given that CapsNet and other deep models are computationally expensive, efforts can be made to develop compressed or pruned versions for faster inference, especially for deployment on edge devices in remote or resource-constrained healthcare settings.

9.3 Applications

1. **Early Diagnosis of Alzheimer's Disease :** The primary application of this project is to aid in the early detection and accurate staging of Alzheimer's disease. By analyzing MRI scans, the system can help clinicians identify subtle brain changes associated with the disease before noticeable symptoms appear, enabling earlier intervention and better treatment planning.

2. **Longitudinal Monitoring :** The system could be expanded to support longitudinal studies where it monitors patients over time. By analyzing a series of MRI scans, the system could track the progression of Alzheimer's disease, providing insights into how the disease evolves and helping in the assessment of treatment effectiveness.

3. **Academic Research :** This project can be used to study and compare the learning capabilities of various deep learning architectures on complex medical imaging data. By analyzing performance across CNN, VGG16, ResNet50, and CapsNet, researchers can gain insights into how different models capture spatial and structural features. It serves as a valuable benchmark for developing more effective models in medical image classification.

CHAPTER 10
APPENDIX

10.1 Appendix A

10.1.1 Problem Statement Feasibility Assessment

The core problem addressed in this project is the automated classification of brain MRI scans for the detection and staging of Alzheimer's disease using deep learning models. From a computational standpoint, this problem can be framed and evaluated in terms of its complexity class and satisfiability under modern computational theory.

10.1.2 Satisfiability Analysis

The feasibility of the problem is analyzed using Boolean Satisfiability (SAT) principles, where the objective is to determine whether a given input (brain MRI scan) can be correctly classified by a function (deep learning model) under a predefined set of constraints (e.g., stage labels, structural features). The problem can be abstracted as:

$$f(x) = y \quad suchthat \quad f \in \mathcal{F}, \quad x \in X, \quad y \in \{0, 1, 2\} \tag{10.1}$$

Here, $\mathcal{F}$ represents the space of all feasible deep learning models (CNN, VGG16, ResNet, CapsNet), X is the domain of preprocessed MRI scan inputs, and y denotes the predicted Alzheimer's stage (e.g., normal, mild, severe). The satisfiability question involves whether there exists an $f \in \mathcal{F}$ such that all MRI inputs $x_i \in X$ map to their true corresponding labels y_i with sufficiently low error.

10.1.3 Complexity Class Assessment

From a theoretical perspective, the classification task using deep learning can be seen as a decision problem: determining whether a certain MRI scan x belongs to a given class y. This is a verifiable problem in polynomial time (P), given that once a model is trained, inference (classification) takes $O(n)$ time for each input. Thus, the inference component of our problem lies within class **P**.

However, the **training** process involves optimizing a non-convex loss function over a high-dimensional parameter space, which is known to be NP-Hard in general. Formally, finding a global minimum of the loss function $L(\theta)$ for a deep neural network can be modeled as:

$$\min_{\theta \in \Theta} L(\theta; X, Y) \tag{10.2}$$

Due to the existence of multiple local minima and saddle points, no polynomial-time algorithm is guaranteed to find the global minimum. Hence, training deep networks, especially for high-dimensional medical imaging tasks, is theoretically NP-Hard.

10.1.4 Relevance of Modern Algebra and Mathematical Models

Algebraic structures such as vector spaces, tensor operations, and group transformations play a central role in deep learning, especially in convolution operations, weight optimization, and model representation. MRI data $x \in R^{H \times W \times C}$ can be treated as a tensor in a vector space, and the neural network functions are mappings:

$$f : R^{H \times W \times C} \rightarrow R^k \tag{10.3}$$

where k is the number of Alzheimer's stages. The convolutional and capsule transformations rely on algebraic principles of linear transformations, matrix multiplication, and high-dimensional mapping.

10.1.5 Feasibility Conclusion

While the training of deep learning models is an NP-Hard optimization problem, the feasibility of this project is ensured through the use of heuristic and approximate algorithms (e.g., gradient descent, backpropagation) that yield satisfactory results in practice. Moreover, the inference stage is in class **P**, which guarantees efficient real-time prediction of Alzheimer's disease stages. Therefore, the overall problem is computationally feasible using current machine learning methods and mathematical frameworks.

10.2 Appendix B

10.2.1 Survey Paper Details

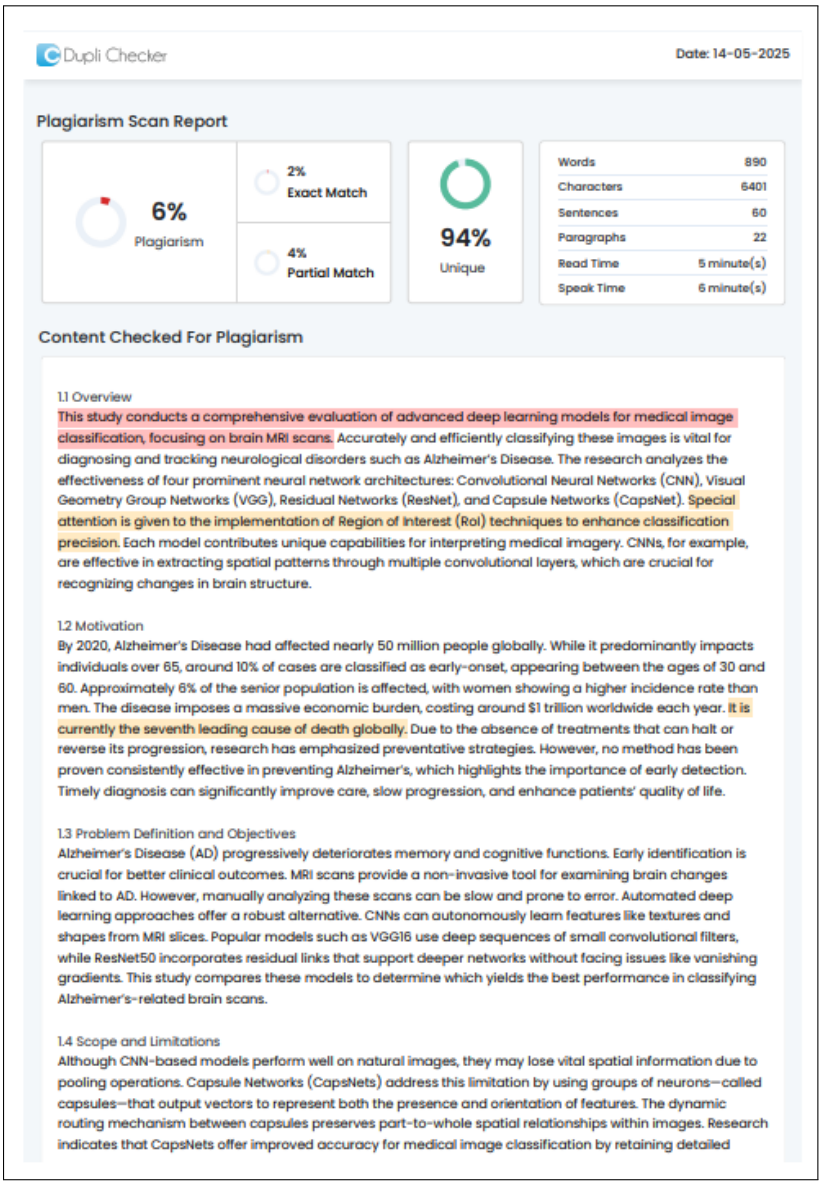
- **Paper Title:** Comparative Analysis of Deep learning algorithms for Clinical Diagnosis
- **Name of Journal:** Bulletin for Technology and History
- **Page Numbers:** 14
- **DOI:** DOI:10.37326/bthnlv24.11/1851
- **Paper Status:** Published

10.2.2 Implementation Paper Details

- **Paper Title:** Image Classification and Processing for Alzheimer's Disease Detection using Deep Learning Algorithms
- **Name of Conference:** 3rd International Conference on Artificial Intelligence: Theory and Applications (AITA 2025)
- **Paper Status:** Applied

10.3 Appendix C

The plagiarism report for the project report has been attached herewith.
Similarity: 6%



Dupli Checker — Date: 14-05-2025

Plagiarism Scan Report

6% Plagiarism	2% Exact Match; 4% Partial Match
94% Unique	

Words	890
Characters	6401
Sentences	60
Paragraphs	22
Read Time	5 minute(s)
Speak Time	6 minute(s)

Content Checked For Plagiarism

1.1 Overview

This study conducts a comprehensive evaluation of advanced deep learning models for medical image classification, focusing on brain MRI scans. Accurately and efficiently classifying these images is vital for diagnosing and tracking neurological disorders such as Alzheimer's Disease. The research analyzes the effectiveness of four prominent neural network architectures: Convolutional Neural Networks (CNN), Visual Geometry Group Networks (VGG), Residual Networks (ResNet), and Capsule Networks (CapsNet). Special attention is given to the implementation of Region of Interest (RoI) techniques to enhance classification precision. Each model contributes unique capabilities for interpreting medical imagery. CNNs, for example, are effective in extracting spatial patterns through multiple convolutional layers, which are crucial for recognizing changes in brain structure.

1.2 Motivation

By 2020, Alzheimer's Disease had affected nearly 50 million people globally. While it predominantly impacts individuals over 65, around 10% of cases are classified as early-onset, appearing between the ages of 30 and 60. Approximately 6% of the senior population is affected, with women showing a higher incidence rate than men. The disease imposes a massive economic burden, costing around $1 trillion worldwide each year. It is currently the seventh leading cause of death globally. Due to the absence of treatments that can halt or reverse its progression, research has emphasized preventative strategies. However, no method has been proven consistently effective in preventing Alzheimer's, which highlights the importance of early detection. Timely diagnosis can significantly improve care, slow progression, and enhance patients' quality of life.

1.3 Problem Definition and Objectives

Alzheimer's Disease (AD) progressively deteriorates memory and cognitive functions. Early identification is crucial for better clinical outcomes. MRI scans provide a non-invasive tool for examining brain changes linked to AD. However, manually analyzing these scans can be slow and prone to error. Automated deep learning approaches offer a robust alternative. CNNs can autonomously learn features like textures and shapes from MRI slices. Popular models such as VGG16 use deep sequences of small convolutional filters, while ResNet50 incorporates residual links that support deeper networks without facing issues like vanishing gradients. This study compares these models to determine which yields the best performance in classifying Alzheimer's-related brain scans.

1.4 Scope and Limitations

Although CNN-based models perform well on natural images, they may lose vital spatial information due to pooling operations. Capsule Networks (CapsNets) address this limitation by using groups of neurons—called capsules—that output vectors to represent both the presence and orientation of features. The dynamic routing mechanism between capsules preserves part-to-whole spatial relationships within images. Research indicates that CapsNets offer improved accuracy for medical image classification by retaining detailed

Figure 10.1: Plagiarism Report

CHAPTER 11
REFERENCES

[1] S. Albawi, T. A. Mohammed and S. Al-Zawi, "Understanding of a convolutional neural network," 2017 International Conference on Engineering and Technology (ICET), Antalya, Turkey, 2017, pp. 1-6, doi: 10.1109/ICEngTechnol.2017.8308186.

[2] O'Shea, Keiron Nash, Ryan. (2015). An Introduction to Convolutional Neural Networks. ArXiv e-prints.

[3] Alzubaidi, L., Zhang, J., Humaidi, A.J. et al. Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. J Big Data 8, 53 (2021). https://doi.org/10.1186/s40537-021-00444-8

[4] F. Altaf, S. M. S. Islam, N. Akhtar and N. K. Janjua, "Going Deep in Medical Image Analysis: Concepts, Methods, Challenges, and Future Directions," in IEEE Access, vol. 7, pp. 99540-99572, 2019, doi: 10.1109/ACCESS.2019.2929365.

[5] Mittal, Ansh Kumar, Deepika. (2019). AiCNNs (Artificially-integrated Convolutional Neural Networks) for Brain Tumor Prediction. EAI Endorsed Transactions on Pervasive Health and Technology. 5. 10.4108/eai.12-2-2019.161976.

[6] Klang E. Deep learning and medical imaging. J Thorac Dis. 2018 Mar;10(3):1325-1328. doi: 10.21037/jtd.2018.02.76. PMID: 29708147; PMCID: PMC5906243.

[7] Li M, Jiang Y, Zhang Y, Zhu H. Medical image analysis using deep learning algorithms. Front Public Health. 2023 Nov 7;11:1273253. doi: 10.3389/fpubh.2023.1273253. PMID: 38026291; PMCID: PMC10662291.

[8] Haq, Mahmood Ul Sethi, Muhammad Rehman, Atiq. (2023). Capsule Network with Its Limitation, Modification, and Applications—A Survey. Machine Learning and Knowledge Extraction. 5. 891-921. 10.3390/make5030047.

[9] Tran, Minh Vo, Khoa Quinn, Kyle Nguyen, Hien Luu, Khoa Le, Ngan. (2022). CapsNet for Medical Image Segmentation. 10.48550/arXiv.2203.08948.

[10] Barragán-Montero A, Javaid U, Valdés G, Nguyen D, Desbordes P, Macq B, Willems S, Vandewinckele L, Holmström M, Löfman F, Michiels S, Souris K, Sterpin E, Lee JA. Artificial intelligence and machine learning for medical imaging: A technology review. Phys Med. 2021

Mar;83:242-256. doi: 10.1016/j.ejmp.2021.04.016. Epub 2021 May 9. PMID: 33979715; PMCID: PMC8184621.

[11] Mazzia, V., Salvetti, F. Chiaberge, M. Efficient-CapsNet: capsule network with self-attention routing. Sci Rep 11, 14634 (2021). https://doi.org/10.1038/s41598-021-93977-0

[12] Liang, Jiazhi. (2020). Image classification based on RESNET. Journal of Physics: Conference Series. 1634. 012110. 10.1088/1742-6596/1634/1/012110.

[13] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 770-778, doi: 10.1109/CVPR.2016.90.

[14] Simonyan, Karen Zisserman, Andrew. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv 1409.1556.

[15] Tammina, Srikanth. (2019). Transfer learning using VGG-16 with Deep Convolutional Neural Network for Classifying Images. International Journal of Scientific and Research Publications (IJSRP). 9. p9420. 10.29322/IJSRP.9.10.2019.p9420.

[16] Mukhometzianov, Rinat Carrillo, Juan. (2018). CapsNet comparative performance evaluation for image classification. 10.48550/arXiv.1805.11195.

[17] I. J. Goodfellow et al., "Multi-digit number recognition from street view imagery using deep convolutional neural networks," arXiv preprint arXiv:1312.6082, 2013.

[18] G. E. Hinton, "Shape representation in parallel systems," in *International Joint Conference on Artificial Intelligence*, vol. 2, 1981.

[19] G. E. Hinton, A. Krizhevsky, and S. D. Wang, "Transforming auto-encoders," in *International Conference on Artificial Neural Networks*, pp. 44–51, Springer, 2011.

[20] M. Jaderberg et al., "Spatial transformer networks," in *Advances in Neural Information Processing Systems*, pp. 2017–2025, 2015.

[21] Y. Netzer et al., "Reading digits in natural images with unsupervised feature learning," in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, vol. 2011, p. 5, 2011.

[22] B. A. Olshausen, C. H. Anderson, and D. C. Van Essen, "A neurobiological model of visual attention and invariant pattern recognition based on dynamic routing of information," *Journal of Neuroscience*, 13(11):4700–4719, 1993.

[23] L. Wan et al., "Regularization of neural networks using dropconnect," in *Proceedings of the 30th International Conference on Machine Learning (ICML-13)*, pp. 1058–1066, 2013.

[24] M. D. Zeiler and R. Fergus, "Stochastic pooling for regularization of deep convolutional neural networks," arXiv preprint arXiv:1301.3557, 2013.

[25] Sara Sabour, Nicholas Frosst, and Geoffrey E. Hinton. Dynamic routing between capsules. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 3856–3866, 2017.

[26] Michael A. Alcorn, Qi Li, Zhitao Gong, Chengfei Wang, Long Mai, Wei-Shinn Ku, and Anh Nguyen. Strike (with) a pose: Neural networks are easily fooled by strange poses of familiar objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4845–4854, 2019.

[27] Patrick M. K., Adekoya A. F., Mighty A. A., and Edward B. Y. Capsule networks – a survey. *Journal of King Saud University - Computer and Information Sciences*, 34(1):1295–1310, 2022.

[28] J. Rajasegaran, V. Jayasundara, S. Jayasekara, H. Jayasekara, S. Seneviratne, and R. Rodrigo. DeepCaps: Going deeper with capsule networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10725–10733, 2019.

Plagiarism Scan Report

6%
Plagiarism

2%
Exact Match

4%
Partial Match

94%
Unique

Words	890
Characters	6401
Sentences	60
Paragraphs	22
Read Time	5 minute(s)
Speak Time	6 minute(s)

Content Checked For Plagiarism

1.1 Overview

This study conducts a comprehensive evaluation of advanced deep learning models for medical image classification, focusing on brain MRI scans. Accurately and efficiently classifying these images is vital for diagnosing and tracking neurological disorders such as Alzheimer's Disease. The research analyzes the effectiveness of four prominent neural network architectures: Convolutional Neural Networks (CNN), Visual Geometry Group Networks (VGG), Residual Networks (ResNet), and Capsule Networks (CapsNet). Special attention is given to the implementation of Region of Interest (RoI) techniques to enhance classification precision. Each model contributes unique capabilities for interpreting medical imagery. CNNs, for example, are effective in extracting spatial patterns through multiple convolutional layers, which are crucial for recognizing changes in brain structure.

1.2 Motivation

By 2020, Alzheimer's Disease had affected nearly 50 million people globally. While it predominantly impacts individuals over 65, around 10% of cases are classified as early-onset, appearing between the ages of 30 and 60. Approximately 6% of the senior population is affected, with women showing a higher incidence rate than men. The disease imposes a massive economic burden, costing around $1 trillion worldwide each year. It is currently the seventh leading cause of death globally. Due to the absence of treatments that can halt or reverse its progression, research has emphasized preventative strategies. However, no method has been proven consistently effective in preventing Alzheimer's, which highlights the importance of early detection. Timely diagnosis can significantly improve care, slow progression, and enhance patients' quality of life.

1.3 Problem Definition and Objectives

Alzheimer's Disease (AD) progressively deteriorates memory and cognitive functions. Early identification is crucial for better clinical outcomes. MRI scans provide a non-invasive tool for examining brain changes linked to AD. However, manually analyzing these scans can be slow and prone to error. Automated deep learning approaches offer a robust alternative. CNNs can autonomously learn features like textures and shapes from MRI slices. Popular models such as VGG16 use deep sequences of small convolutional filters, while ResNet50 incorporates residual links that support deeper networks without facing issues like vanishing gradients. This study compares these models to determine which yields the best performance in classifying Alzheimer's-related brain scans.

1.4 Scope and Limitations

Although CNN-based models perform well on natural images, they may lose vital spatial information due to pooling operations. Capsule Networks (CapsNets) address this limitation by using groups of neurons—called capsules—that output vectors to represent both the presence and orientation of features. The dynamic routing mechanism between capsules preserves part-to-whole spatial relationships within images. Research indicates that CapsNets offer improved accuracy for medical image classification by retaining detailed

Comparative Analysis of Deep learning algorithms for Clinical Diagnosis

Aneesh Tufchi
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, Maharashtra-411043

Shrihari Tiwari
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, Maharashtra-411043

Manshu Khajuria
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, Maharashtra-411043

Kshiteej Pawar
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, Maharashtra-411043

Prof. Virendra Bagade
Asst. Professor, Department of Computer Engineering
Pune Institute of Computer Technology
Pune, Maharashtra-411043

Abstract

This research paper presents a detailed comparative analysis of deep learning models applied to the classification of medical images, specifically focusing on brain MRI scans. Accurate and automated classification of brain MRI images is crucial in diagnosing and staging neurodegenerative diseases such as Alzheimer's, and this study investigates the performance of four cutting-edge deep learning architectures: Convolutional Neural Networks (CNN), Visual Geometry Group Networks (VGG), Residual Networks (ResNet), and Capsule Networks (CapsNet), with an emphasis on the utilization of Region of Interest (RoI) techniques to enhance classification accuracy.

Each of these models brings unique strengths to the task of medical image classification. CNNs, with their layered convolutional operations, excel in learning hierarchical spatial features critical for identifying patterns in brain tissue. VGG networks, known for their deep architecture with small 3x3 filters, allow for more detailed feature extraction, enabling them to capture subtle differences in brain structures associated with various disease stages. ResNet, with its innovative skip connections, overcomes the vanishing gradient problem in deep networks, making it highly effective for learning complex patterns in large, multi-layered architectures. CapsNet, on the other hand, is a relatively new model that maintains the spatial relationships between features, which is particularly important in medical images where anatomical structures vary in orientation and position.

The models are rigorously evaluated across several key metrics, including classification accuracy, precision, recall and F1-score . The study also focuses on the models' ability to generalize across different datasets and on computational efficiency, essential factors when considering real-world clinical deployment. Preprocessing techniques such as image normalization, resizing, and data augmentation (including rotation, flipping, and zooming) are employed to optimize the models' performance and prevent overfitting, particularly in situations where the dataset size is limited.

Through this work, we contribute to the growing body of knowledge in the field of medical image analysis, offering insights into the practical applications of deep learning in healthcare.

Index Terms

CNN ,VGG ,ResNet, CapsNet ,Image Classification, Deep Learning, RoI, GANET , SBO ,ESBO

I. Introduction

In modern medical diagnostics, accurate and efficient image classification has emerged as a critical tool for the early detection and staging of diseases. Among various medical imaging modalities, brain MRI (Magnetic Resonance Imaging) plays a vital role in diagnosing neurodegenerative disorders such as Alzheimer's disease, brain tumors, and multiple sclerosis. These diseases often present complex patterns of structural changes in the brain, making manual interpretation challenging and time-consuming, especially when attempting to detect early-stage abnormalities that may be subtle and difficult to identify. Therefore, the need for automated and precise image classification systems has become increasingly pressing, as these systems can significantly improve diagnostic accuracy and reduce the workload on radiologists.

The application of deep learning techniques to medical image analysis, particularly for brain MRI scans, has gained substantial momentum in recent years. Deep learning, a subfield of machine learning, has revolutionized image recognition tasks by enabling

models to automatically learn hierarchical features from raw image data, eliminating the need for manual feature extraction. These models, when applied to medical images, can detect patterns that may not be immediately apparent to the human eye, leading to improved diagnosis, staging, and prognosis of various brain-related diseases.

Among the deep learning models, Convolutional Neural Networks (CNNs) are particularly well-suited for image classification due to their ability to capture spatial hierarchies in data. CNNs have been widely used for various medical imaging tasks, including lesion detection, tumor segmentation, and classification of brain diseases. However, as medical images, especially brain MRIs, present unique challenges such as the presence of noise, anatomical variations, and limited datasets, more advanced CNN architectures are necessary to improve classification accuracy and generalization.

VGG (Visual Geometry Group) networks, a deep convolutional network architecture, have proven effective in various image classification tasks. Their use of small filters (e.g., 3x3 kernels) and deep structures allows for more detailed feature extraction, capturing minute variations in medical images that could indicate early-stage diseases. VGG's depth enables it to detect fine-grained patterns, making it particularly useful for classifying complex medical datasets where small differences in image features can have significant diagnostic implications.

Residual Networks (ResNet) take deep learning a step further by addressing the problem of vanishing gradients, which commonly occurs in very deep networks. The introduction of residual connections allows the model to bypass certain layers, ensuring that the gradient flows effectively through the network. This feature is particularly important in medical image classification, where learning intricate, high-level features across many layers can be essential for accurate disease staging. ResNet's ability to train deep networks without performance degradation has made it one of the most successful architectures in medical image classification, especially for tasks that require distinguishing between early and advanced stages of diseases like Alzheimer's.

Capsule Networks (CapsNet) are an advanced form of neural networks designed to overcome the limitations of traditional Convolutional Neural Networks (CNNs). They were introduced by Sabour and Hinton in 2017, primarily focusing on tasks such as image classification and object detection. Unlike CNNs, which struggle with recognizing spatial hierarchies and relationships between parts of objects, CapsNet aims to model these relationships, improving generalization and recognition capabilities. For instance, a CNN might misclassify a jumbled bicycle with scattered wheels and handlebars as a bicycle, focusing only on the presence of the parts. In contrast, a CapsNet can learn the spatial relationships between those parts (like how the wheels should be positioned relative to the frame) and correctly identify that the object is not a properly assembled bicycle. Basically, a capsule network is a type of neural network that performs inverse graphics. For example, in object detection, the object is divided into subparts. To represent that object, a hierarchical relationship is developed between all subparts. The implementation of CapsNet is divided into three main parts. These parts are the input layer, hidden layer, and output layer.
In terms of security, capsule networks have been shown to be more robust against certain types of attacks compared to traditional CNNs.

The goal of this research is to evaluate and compare the performance of these four prominent deep learning architectures—CNN, VGG, ResNet, and CapsNet—on the task of classifying brain MRI images. We focus on key performance metrics such as accuracy, precision, recall and F1-score as well as the models' ability to generalize across different datasets. In addition to these metrics, computational efficiency is considered, as real-time diagnostic applications require not only accurate but also fast and resource-efficient models.

The importance of this research lies in its potential to advance automated medical diagnostics, particularly in the early detection and staging of neurodegenerative diseases. By providing a thorough comparison of different deep learning models, this study aims to highlight the strengths and limitations of each model and provide insights into which architectures are best suited for various types of medical image classification tasks. Furthermore, the findings from this research could pave the way for the integration of deep learning-based image classification systems in clinical practice, ultimately improving patient outcomes by enabling more accurate and timely diagnoses.

II. ROI Techniques in CNNs

A. ROI Pooling

ROI pooling is a technique designed to handle variable-sized input regions in CNNs, particularly useful for object detection frameworks like Faster R-CNN. The key steps involved in ROI pooling are:

- **ROI Definition:** An ROI is defined by a bounding box with coordinates $(x_{min}, y_{min}, x_{max}, y_{max})$.

- **Binning:** The bounding box is divided into a fixed number of bins (e.g., $H \times W$).
- **Max Pooling:** Max pooling is applied to each bin to generate a fixed-size output feature map.

The mathematical formulation for ROI pooling can be expressed as:

$$F_{ROI} = \text{Pooling}(F_{feature}, ROI) \tag{1}$$

where $F_{feature}$ is the feature map from the last convolutional layer.

B. ROI Align

ROI Align improves upon the limitations of ROI pooling by avoiding quantization errors. It performs bilinear interpolation on the feature map to ensure a better alignment with the original image pixels. This approach is critical for tasks requiring high precision, such as instance segmentation.

The mathematical representation of ROI align is given by:

$$F_{ROI} = \text{ROIAlign}(F_{feature}, ROI) \tag{2}$$

where the output size is maintained without introducing artifacts due to binning.

C. Implementation in VGG16

VGG16, characterized by its deep architecture consisting of 16 layers, employs ROI pooling as follows:

- **Feature Extraction:** Convolutional layers extract high-level features from the input image.
- **ROI Pooling:** After the last convolutional layer, ROIs are pooled into a fixed-size feature map.
- **Classification:** The pooled features are fed into fully connected layers for classification.

The ROI pooling step can be mathematically represented as:

$$F_{ROI} = \max_{i,j}(F_{feature}(x_i, y_j)) \quad \forall (x_i, y_j) \in ROI \tag{3}$$

This allows the model to effectively learn from localized features relevant to the defined ROIs.

D. Implementation in ResNet

ResNet introduces skip connections that alleviate the vanishing gradient problem, facilitating the training of deeper networks. The integration of ROIs in ResNet follows a similar flow:

- **Feature Extraction:** Features are extracted through several residual blocks.
- **ROI Align:** The ROI align method is applied to generate a fixed-size output feature map from the selected ROIs.
- **Classification and Regression:** The pooled features are then used for both classification and bounding box regression.

The use of ROI align can be expressed as:

$$F_{ROI} = \sum_{i=1}^{N} \text{BilinearInterpolate}(F_{feature}, (x_i, y_i)) \tag{4}$$

where N is the number of sampled points from the ROI, leading to a more accurate representation of the region.

III. ROI Segmentation

Region of Interest (ROI) Segmentation refers to the process of identifying and delineating specific regions within an image that are deemed significant for further analysis. This technique is crucial in various applications, including medical imaging, autonomous driving, and object detection. ROI segmentation allows for a more focused approach, improving both the accuracy and efficiency of subsequent tasks.

A. Mathematical Framework

The process of ROI segmentation can be mathematically framed as follows:

- **Input Representation:** Let I represent the input image and ROI be the region of interest defined by coordinates $(x_{min}, y_{min}, x_{max}, y_{max})$.
- **Feature Extraction:** The feature map $F_{feature}$ is generated through a series of convolutional layers:

$$F_{feature} = \text{CNN}(I) \tag{5}$$

- **ROI Pooling or Align:** After feature extraction, either ROI Pooling or ROI Align is applied:

$$F_{ROI} = \text{ROIAlign}(F_{feature}, ROI) \tag{6}$$

Here, F_{ROI} is the pooled feature map corresponding to the ROI.

- **Segmentation Mask Generation:** The segmentation mask S can be obtained using a pixel-wise classification approach, where each pixel in the ROI is assigned a label:

$$S = \text{argmax}(F_{segmentation}(F_{ROI})) \tag{7}$$

This function $F_{segmentation}$ represents a classifier that outputs probabilities for each class over the features extracted from the ROI.

B. Techniques Used in ROI Segmentation

- **Convolutional Neural Networks (CNNs):** CNNs are typically employed to extract spatial hierarchies of features from images. The architecture can include various layers, such as convolutional, pooling, and fully connected layers, tailored to segment objects from ROIs effectively.
- **Semantic Segmentation:** In semantic segmentation, every pixel is classified into a category. This is useful when the objective is to identify and delineate all objects within an ROI. Techniques like U-Net or Fully Convolutional Networks (FCNs) can be utilized for this purpose.
- **Instance Segmentation:** Unlike semantic segmentation, instance segmentation differentiates between distinct objects of the same class. Models like Mask R-CNN extend traditional detection frameworks to include a segmentation mask for each detected instance. The output can be defined as:

$$M_i = F_{mask}(F_{ROI}) \quad \forall i \in \text{instances} \tag{8}$$

where M_i is the mask for the i-th instance.

IV. System Architechture

The system architecture for classifying medical images, specifically brain MRI scans, using deep learning models, consists of several interconnected layers that work together to preprocess the data, extract features, apply machine learning models, and present results. Each of these layers has a distinct role in ensuring the accuracy, efficiency, and scalability of the image classification system. Below is a detailed breakdown of the system architecture.

A. Data Collection Layer

The data collection layer forms the foundation of the image classification system. Medical images, particularly brain MRI scans, are sourced from a variety of repositories and databases, which could include:

1) Public datasets: Examples include the Alzheimer's Disease Neuroimaging Initiative (ADNI) or Medical Image Computing and Computer-Assisted Intervention (MICCAI) challenges, which provide labeled MRI images of healthy individuals and patients in various stages of neurodegenerative diseases.

2) Hospital databases: Clinically gathered data from hospitals, which may contain anonymized MRI scans of patients along with corresponding medical history and disease staging.

These datasets often contain labeled images representing different stages of diseases such as Alzheimer's (e.g., Mild Cognitive Impairment (MCI), Early Alzheimer's Disease (AD), or Healthy Control (HC)). The labeled data is crucial for supervised learning models like CNN, VGG, ResNet, and CapsNet. Challenges at this stage:

3) Data diversity: Datasets must cover various demographic groups, imaging techniques, and disease stages to ensure the models generalize well across different patient populations.

4) Data size: Medical datasets can be relatively small due to privacy concerns and the cost of collecting and annotating medical images, leading to issues such as overfitting. Therefore, augmentation and transfer learning are often necessary.

B. Data Preprocessing Layer

Once the MRI images are collected, they undergo several preprocessing steps to prepare them for input into the deep learning models. The data preprocessing layer is critical because the quality of the input data directly influences the performance of the models. Key tasks in this layer include:

1) Resizing: MRI images come in various sizes depending on the scanning equipment used. For consistency, all images are resized to a fixed resolution (e.g., 224x224 or 256x256 pixels). This standardization ensures uniform input dimensions for the models.

2) Normalization: The pixel values of MRI images can vary widely, depending on the scanning intensity. Normalization transforms these values into a standard range, such as [0,1] or [-1,1], which helps the neural network converge more quickly and improves numerical stability during training.

3) Data Augmentation: To combat the relatively small size of medical image datasets, data augmentation techniques are applied to artificially expand the dataset. Common augmentations include: o Rotation: Rotating the images by random angles to help the model learn rotational invariance. o Flipping: Horizontally or vertically flipping the images to create new training samples. o Zooming: Randomly zooming in and out to teach the model to handle images at different scales. o Shifting: Translating the images slightly to simulate variability in patient positioning during the MRI scan.

4) Image Slicing: MRI scans are typically 3D images composed of multiple 2D slices taken at different depths. Depending on the approach, these 2D slices can be used individually, or the 3D volume can be processed as a whole. For some deep learning models, individual slices are classified independently, while for others (such as 3D CNNs), the entire volume is processed to capture spatial relationships.

These preprocessing techniques ensure that the images are clean, standardized, and augmented, improving the models' ability to generalize and perform well on unseen data.

C. *Feature Engineering*

In traditional machine learning, feature engineering involves manually selecting and transforming features to improve model performance. However, in deep learning, particularly with CNNs and their variants, feature extraction is handled automatically by the convolutional layers. These layers learn to identify low-level features (e.g., edges, textures) in the early stages and high-level features (e.g., anatomical structures, patterns related to disease) in the deeper layers.

For brain MRI classification, feature extraction focuses on:

1) Identifying anatomical regions of interest (ROIs): The network learns to focus on key brain regions like the hippocampus, which is known to exhibit atrophy in Alzheimer's patients.

2) Capturing spatial relationships: Especially important for CapsNet, which preserves the pose and orientation of features, ensuring that spatial hierarchies are maintained.

For models like CNN, VGG, ResNet, and CapsNet, the convolutional layers and pooling layers work together to create hierarchical feature maps, which capture increasing levels of abstraction. In this process:

- Convolutional filters slide over the input image to detect features, producing feature maps.
- Pooling layers downsample the feature maps, reducing their size while retaining the most important information. This helps in reducing computational complexity and preventing overfitting.

In medical image classification, feature extraction is particularly challenging because the features that differentiate disease stages may be subtle, requiring deeper architectures (such as VGG or ResNet) to capture these variations effectively.

D. *Modeling Layer (Machine Learning)*

The modeling layer is where the deep learning architectures are applied to the preprocessed and feature-extracted data to perform classification. The models used in this study are:

1) Convolutional Neural Networks (CNNs): A standard architecture for image classification, CNNs use convolutional layers to detect features and pooling layers to reduce spatial dimensions. CNNs are widely used in medical imaging because of their ability to handle large amounts of spatial data efficiently. The network learns to extract relevant features from MRI scans and classify them into disease stages.

- The convolutional layer applies a set of filters (or kernels) to the input image. For a given input image X, the output feature map Y after applying the filter W can be expressed as:

$$Y_{i,j,k} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} W_{m,n,k} \cdot X_{i+m,j+n,c} + b_k \tag{9}$$

  Where:
  - $Y_{i,j,k}$ is the output feature map at position (i, j) for the k-th filter.
  - $W_{m,n,k}$ is the weight of the k-th filter at position (m, n).
  - $X_{i+m,j+n,c}$ is the input value at position $(i + m, j + n)$ of the c-th channel.
  - b_k is the bias associated with the k-th filter.
  - M and N are the dimensions of the filter.

- Activation Function: After the convolution operation, an activation function (commonly ReLU) is applied element-wise:

$$Z_{i,j,k} = \text{ReLU}(Y_{i,j,k}) = \max(0, Y_{i,j,k}) \tag{10}$$

  Where:
  - $Z_{i,j,k}$ is the output after the activation function.

- Pooling Layer: Pooling layers reduce the spatial dimensions of the feature maps, typically using max pooling. The output after max pooling can be expressed as:

$$P_{i,j,k} = \max_{m,n} Z_{s \cdot i+m, s \cdot j+n, k} \tag{11}$$

  Where:
  - $P_{i,j,k}$ is the pooled output.
  - s is the stride used for pooling.
  - m and n are the indices that iterate over the pooling window.

- Fully Connected Layer The output of the final pooling layer is flattened and fed into a fully connected layer. The output of this layer is calculated as:

$$O_j = \sum_{i=1}^{N} W_{ij} \cdot A_i + b_j \tag{12}$$

  Where:
  - O_j is the output of the j-th neuron in the fully connected layer.
  - W_{ij} is the weight from neuron i to neuron j.
  - A_i is the activation from the previous layer (flattened feature maps).
  - b_j is the bias term for the j-th neuron.
  - N is the number of inputs to the fully connected layer.

- Output Layer For classification tasks, the final output layer often uses the softmax function to convert the logits into probabilities:

$$P(y_k) = \frac{e^{O_k}}{\sum_{j=1}^{K} e^{O_j}} \tag{13}$$

  Where:
  - $P(y_k)$ is the probability of class k.
  - O_k is the output of the k-th neuron in the output layer.
  - K is the number of classes.

2) VGG: VGG extends CNNs by utilizing deeper networks, primarily with small 3×3 convolutional filters. It uses max pooling layers with a fixed stride of 2 and a 2x2 pool size, which helps to reduce the spatial dimensions while maintaining the critical features. This increased depth allows the model to capture more complex and finer details in MRI images. VGG typically consists of multiple convolutional layers followed by max pooling layers, with three fully connected layers at the end. The first two fully connected layers have 4096 neurons each, while the last corresponds to the number of classes in the classification task. It Primarily employs the ReLU (Rectified Linear Unit) activation function after each convolutional layer, which has become standard in many modern architectures.

3) ResNet Residual Networks: It solves the problem of vanishing gradients in deep networks by introducing skip connections, which allow gradients to flow directly through the network without degradation. ResNet is particularly effective in medical image classification tasks because it enables the model to learn deep, complex representations of brain structures that differentiate healthy brains from diseased ones.
The core building block of ResNet is the residual block, which adds the input of the block to the output of the convolutional layers, promoting efficient training and deeper architectures.

- Input Layer The input to the ResNet model is typically an image represented as a tensor X of dimensions $H \times W \times C$, where H is height, W is width, and C is the number of channels (3 for RGB images).
- Convolutional Layer The first layer in ResNet usually consists of a convolutional layer that applies a set of filters (kernels) to the input image. The output feature map Y after applying a filter W can be expressed as:

$$Y_{i,j,k} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} W_{m,n,k} \cdot X_{i+m,j+n,c} + b_k \tag{14}$$

  Where:
  - $Y_{i,j,k}$ is the output feature map at position (i, j) for the k-th filter.
  - $W_{m,n,k}$ is the weight of the k-th filter at position (m, n).
  - $X_{i+m,j+n,c}$ is the input value at position $(i + m, j + n)$ of the c-th channel.
  - b_k is the bias associated with the k-th filter.
  - M and N are the dimensions of the filter (typically 3×3 in ResNet).
- Residual Block The core building block of ResNet is the residual block, which enables the network to learn residual functions. A basic residual block can be defined as follows:

$$F(X) = \text{ReLU}(W_2 * \text{ReLU}(W_1 * X + b_1) + b_2) \tag{15}$$

  Where:
  - X is the input to the block.
  - W_1 and W_2 are the weights for the two convolutional layers.
  - b_1 and b_2 are the corresponding biases.

  The output of the block is the sum of the input and the output of the convolutional layers:

$$Z = F(X) + X \tag{16}$$

  Where:
  - Z is the final output of the residual block.
  - The addition $F(X) + X$ represents the skip connection that helps the gradient flow during backpropagation.
- Activation Function After computing the residual output, a non-linear activation function, typically ReLU, is applied to Z:

$$Z' = \text{ReLU}(Z) \tag{17}$$

- Pooling Layer Pooling layers are used in ResNet to reduce the spatial dimensions of the feature maps. The max pooling operation can be represented as:

$$P_{i,j,k} = \max_{m,n} Z_{s \cdot i+m, s \cdot j+n, k'} \tag{18}$$

  Where:
  - $P_{i,j,k}$ is the pooled output.
  - s is the stride used for pooling.
- Fully Connected Layers The output from the final convolutional layer is flattened and passed to a fully connected layer. The output of this layer can be computed as:

$$O_j = \sum_{i=1}^{N} W_{ij} \cdot A_i + b_j \tag{19}$$

  Where:
  - O_j is the output of the j-th neuron in the fully connected layer.
  - W_{ij} is the weight from neuron i to neuron j.
  - A_i is the activation from the previous layer.
  - b_j is the bias term for the j-th neuron.
  - N is the number of inputs to the fully connected layer.

- Output Layer For multi-class classification tasks, the output layer typically uses the softmax function to convert logits into probabilities:

$$P(y_k) = \frac{e^{O_k}}{\sum_{j=1}^{K} e^{O_j}} \tag{20}$$

Where:
  - $P(y_k)$ is the probability of class k.
  - O_k is the output of the k-th neuron in the output layer.
  - K is the total number of classes.

4) Capsule Networks (CapsNet): Unlike CNNs, which are prone to losing spatial information as features are pooled, CapsNet preserves the spatial hierarchy of features using capsules—small groups of neurons that learn not just the presence of a feature but also its orientation, position, and scale. CapsNet's dynamic routing between capsules enables it to handle variations in the input image better, making it especially useful for medical images where subtle differences in orientation and structure are diagnostically significant.

- The mathematical model of a Capsule Network (CapsNet) starts with an input, typically an image or a set of images, that is processed by a convolutional layer. This paper outlines the key equations that describe the information flow through a CapsNet.

$$z_{ij,k} = \sum_{l=1}^{L} W_{ij,k,l} X_{ij,l} + b_{ij,k} \tag{21}$$

where $z_{ij,k}$ is the activation at position (i, j) in the k-th feature map, $W_{ij,k,l}$ is the weight associated with the i-th input channel, $X_{ij,l}$ is the input, and $b_{ij,k}$ is the bias term.
Next, we move to the primary capsule layer, where the output vector of the k-th capsule is given by:

$$V_{ij,k} = \text{Squash}\left(\sum_{l=1}^{L'} W_{ij,k,l} u_{ij,l}\right) \tag{22}$$

The "Squash" function ensures the vector length is between 0 and 1.
Higher-level capsules aggregate inputs from lower-level capsules using routing by agreement, defined as:

$$S_j = \sum_i c_{ij} \hat{u}_{j|i} \tag{23}$$

Here, S_j is the output of the j-th higher-level capsule, c_{ij} is the coupling coefficient, and $\hat{u}_{j|i}$ is the predicted output from lower-level capsules.
The final capsule output is given by:

$$y_k = \text{squash}(s_k) \tag{24}$$

where y_k is the output vector of the k-th capsule, and s_k is its input vector.
These equations illustrate how information flows through the capsule network. Backpropagation is utilized to optimize weights and biases. The network design can be adapted for specific tasks by introducing new routing algorithms or adding layers to enhance speed and interaction between capsules.

E. Optimization Algorithms

To improve model performance and convergence speed, optimization algorithms such as Gannet, SBO (Sequential Bayesian Optimization), and ESBO (Enhanced Sequential Bayesian Optimization) are employed. These algorithms help fine-tune hyper-parameters (like learning rates, batch sizes, and network architecture choices) to maximize model performance on validation datasets

1) Gannet: is designed to explore hyperparameter spaces efficiently, adapting the search process based on past performance to identify optimal configurations.

2) SBO: leverages Bayesian principles to update beliefs about hyperparameter effects dynamically, allowing for a more informed search strategy that improves convergence speed.

3) ESBO: extends SBO by enhancing the search process with advanced sampling techniques, allowing for better exploration of hyperparameter space and yielding improved performance outcomes.
Each model is trained and validated using a portion of the dataset. Cross-validation techniques are employed to assess how well the models generalize to unseen data. Additionally, hyperparameter tuning is performed using the optimization algorithms to optimize learning rates, batch sizes, and other model-specific parameters.

Criteria	**CNN**	**VGG**	**ResNet**	**CapsNet**
Architecture	Multiple convolutional and pooling layers, followed by fully connected layers	Deep, sequential architecture with multiple convolutional layers (16 or 19 layers)	Uses residual blocks that allow the network to skip layers	Uses capsules, groups of neurons that preserve spatial hierarchies
Feature Learning	Learns basic features like edges, textures, and patterns	Focuses on extracting detailed features, larger depth	Residual connections allow better feature propagation	Encodes spatial relationships between features, not just the features
Training Complexity	Easier to train, simpler architecture	High computational cost due to deep layers	Efficient training with residual connections; mitigates vanishing gradient	Difficult to train due to dynamic routing and more complex architecture
Parameter Size	Moderate, depends on the depth of layers	Large, due to depth and fully connected layers	Fewer parameters due to residual connections	Small compared to VGG, but capsules increase the parameter complexity
Training Time	Relatively short, especially for shallow architectures	Long due to the large number of layers and parameters	Efficient due to residual connections despite large depth	Longer due to complex routing mechanisms between capsules
Generalization	Performs well, prone to overfitting if too deep	Good generalization, but prone to overfitting.	Excellent generalization due to residual learning	Better at preserving information,subtle to data distribution

TABLE I
COMPARISON OF DIFFERENT NEURAL NETWORK ARCHITECTURES

V. RESULTS

Measuring Type 1 and Type 2 errors through these metrics allows you to better understand and evaluate the performance of your machine learning models. Depending on the specific use case, different metrics may be prioritized to achieve the desired balance of accuracy and error types In the context of machine learning and statistical classification, Type 1 and Type 2 errors can be measured using specific metrics derived from the confusion matrix. Following the metrics related to these errors:

TYPE 1 ERROR (FALSE POSITIVE)

Definition

A Type 1 error occurs when the model incorrectly identifies a negative instance as positive. This means that the model predicts the presence of a condition (e.g., a disease) when it is not actually present.

Metrics Associated

1) Precision:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Precision indicates the quality of positive predictions. It answers the question: ”Of all instances classified as positive, how many were actually positive?” A high precision means fewer false positives.

2) False Positive Rate (FPR):

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

FPR shows the proportion of actual negatives that were incorrectly identified as positive. It’s also known as the fall-out rate. A low FPR is desirable as it indicates fewer incorrect positive predictions.

3) False Discovery Rate (FDR):

$$\text{FDR} = \frac{\text{FP}}{\text{TP} + \text{FP}}$$

FDR measures the proportion of false positives among all positive predictions. It tells us how many of the predicted positives were actually negative. Lowering the FDR improves the reliability of positive predictions.

TYPE 2 ERROR (FALSE NEGATIVE)

Definition

A Type 2 error occurs when the model fails to identify a positive instance, incorrectly classifying it as negative. This means that the model predicts the absence of a condition when it is actually present.

Metrics Associated

4) Recall (Sensitivity or True Positive Rate):

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Recall measures the model's ability to identify all relevant instances. It answers the question: "Of all actual positives, how many were correctly identified?" A high recall means fewer false negatives.

5) False Negative Rate (FNR):

$$\text{FNR} = \frac{\text{FN}}{\text{TP} + \text{FN}}$$

FNR shows the proportion of actual positives that were incorrectly identified as negative. It's the complement of recall. A low FNR indicates that most positive instances are correctly identified.

GENERAL METRICS (NOT CATEGORIZED AS TYPE 1 OR TYPE 2)

6) Accuracy:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Accuracy measures the overall correctness of the model across all instances. It answers the question: "What proportion of total instances were correctly classified?" While useful, accuracy can be misleading, especially in imbalanced datasets where one class significantly outweighs the other.

7) F1 Score:

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1 score combines precision and recall into a single metric, providing a balance between the two. It's particularly useful when you need to account for both false positives and false negatives. A high F1 score indicates a model that has both good precision and recall.

Measure	**CNN**	**VGG16**	**ResNet**
Accuracy	0.901	0.666	0.529
Precision	0.843	0.625	0.529
Recall	1.0	0.925	1.0
F1 Score	0.915	0.746	0.692
False Negative Rate (FNR)	0.0	0.074	0.0
False Positive Rate (FPR)	0.208	0.625	1.0
False Discovery Rate (FDR)	0.156	0.375	0.470

TABLE II
COMPARISON OF PERFORMANCE METRICS FOR DIFFERENT MODELS

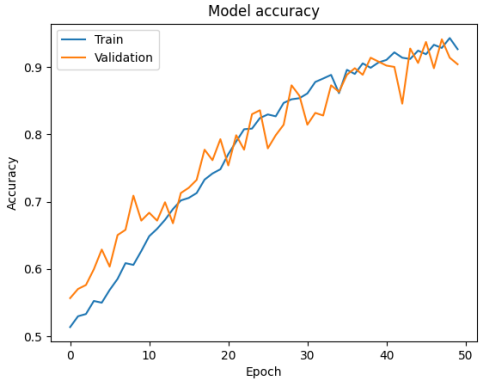


Fig. 1. CNN Accuracy per Epoch

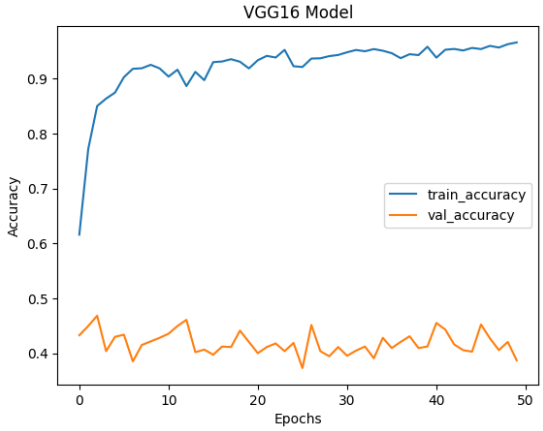


Fig. 2. VGG16 Accuracy per Epoch

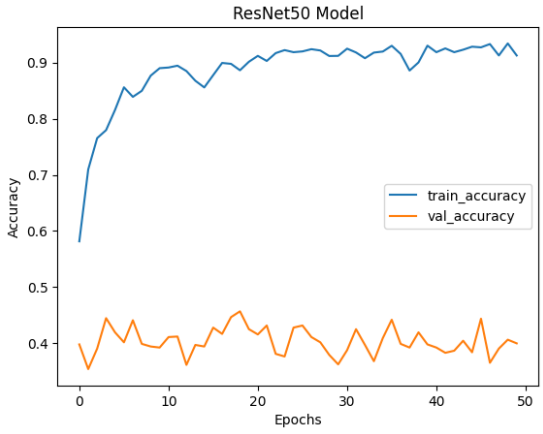


Fig. 3. ResNet Accuracy per Epoch

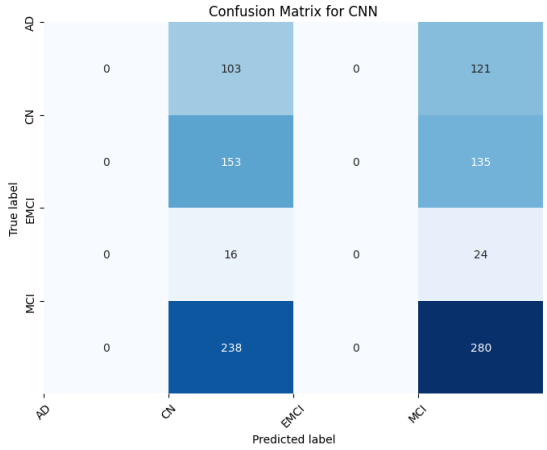


Fig. 4. CNN

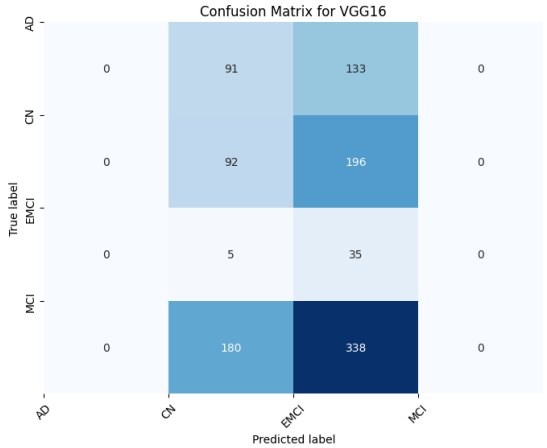


Fig. 5. VGG16

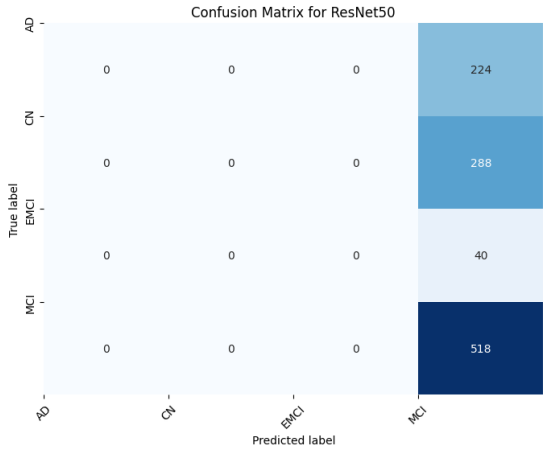


Fig. 6. ResNet

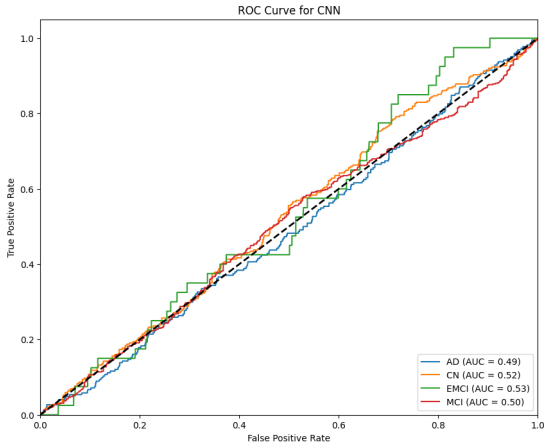


Fig. 7. ROC Curve for CNN

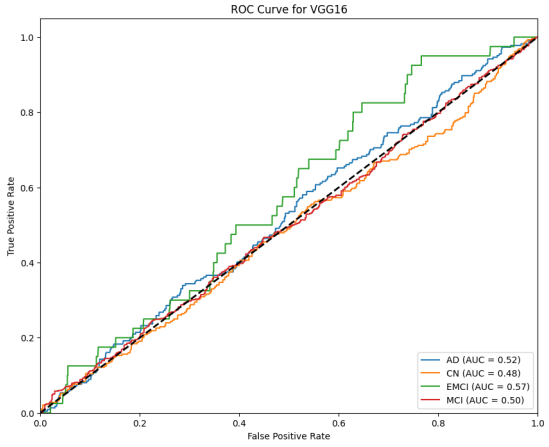


Fig. 8. ROC Curve for VGG16

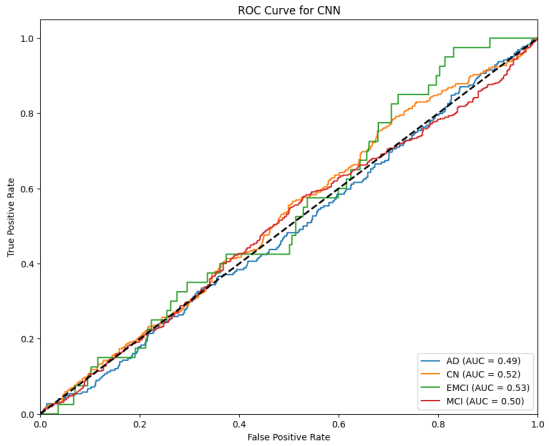


Fig. 9. ROC Curve for ResNet 50

VI. Conclusion

In conclusion, this research presents a comparative analysis of four prominent deep learning models—CNN, VGG, ResNet, and CapsNet—applied to the classification of brain MRI images. Each model demonstrates unique strengths and weaknesses

across different performance metrics such as accuracy, precision, recall, and computational efficiency. CNNs and ResNet stand out for their balance between accuracy and generalization, while VGG exhibits high precision in detailed feature extraction, albeit with higher computational costs. CapsNet, though relatively new, shows promise in preserving spatial hierarchies, making it particularly useful for medical images where subtle anatomical changes are crucial.

The study highlights the importance of model selection based on the specific requirements of clinical applications, such as real-time diagnosis or detailed disease staging. The findings emphasize the potential for integrating these models into real-world healthcare systems, improving diagnostic accuracy and reducing the workload on medical professionals. Future research could explore hybrid models that combine the strengths of multiple architectures, further enhancing the applicability of deep learning in medical imaging.

REFERENCES

[1] S. Albawi, T. A. Mohammed and S. Al-Zawi, "Understanding of a convolutional neural network," 2017 International Conference on Engineering and Technology (ICET), Antalya, Turkey, 2017, pp. 1-6, doi: 10.1109/ICEngTechnol.2017.8308186.

[2] O'Shea, Keiron Nash, Ryan. (2015). An Introduction to Convolutional Neural Networks. ArXiv e-prints.

[3] Alzubaidi, L., Zhang, J., Humaidi, A.J. et al. Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. J Big Data 8, 53 (2021). https://doi.org/10.1186/s40537-021-00444-8

[4] F. Altaf, S. M. S. Islam, N. Akhtar and N. K. Janjua, "Going Deep in Medical Image Analysis: Concepts, Methods, Challenges, and Future Directions," in IEEE Access, vol. 7, pp. 99540-99572, 2019, doi: 10.1109/ACCESS.2019.2929365.

[5] Mittal, Ansh Kumar, Deepika. (2019). AiCNNs (Artificially-integrated Convolutional Neural Networks) for Brain Tumor Prediction. EAI Endorsed Transactions on Pervasive Health and Technology. 5. 10.4108/eai.12-2-2019.161976.

[6] Klang E. Deep learning and medical imaging. J Thorac Dis. 2018 Mar;10(3):1325-1328. doi: 10.21037/jtd.2018.02.76. PMID: 29708147; PMCID: PMC5906243.

[7] Li M, Jiang Y, Zhang Y, Zhu H. Medical image analysis using deep learning algorithms. Front Public Health. 2023 Nov 7;11:1273253. doi: 10.3389/fpubh.2023.1273253. PMID: 38026291; PMCID: PMC10662291.

[8] Haq, Mahmood Ul Sethi, Muhammad Rehman, Atiq. (2023). Capsule Network with Its Limitation, Modification, and Applications—A Survey. Machine Learning and Knowledge Extraction. 5. 891-921. 10.3390/make5030047.

[9] Tran, Minh Vo, Khoa Quinn, Kyle Nguyen, Hien Luu, Khoa Le, Ngan. (2022). CapsNet for Medical Image Segmentation. 10.48550/arXiv.2203.08948.

[10] Barragán-Montero A, Javaid U, Valdés G, Nguyen D, Desbordes P, Macq B, Willems S, Vandewinckele L, Holmström M, Löfman F, Michiels S, Souris K, Sterpin E, Lee JA. Artificial intelligence and machine learning for medical imaging: A technology review. Phys Med. 2021 Mar;83:242-256. doi: 10.1016/j.ejmp.2021.04.016. Epub 2021 May 9. PMID: 33979715; PMCID: PMC8184621.

[11] Mazzia, V., Salvetti, F. Chiaberge, M. Efficient-CapsNet: capsule network with self-attention routing. Sci Rep 11, 14634 (2021). https://doi.org/10.1038/s41598-021-93977-0

[12] Liang, Jiazhi. (2020). Image classification based on RESNET. Journal of Physics: Conference Series. 1634. 012110. 10.1088/1742-6596/1634/1/012110.

[13] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 770-778, doi: 10.1109/CVPR.2016.90.

[14] Simonyan, Karen Zisserman, Andrew. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv 1409.1556.

[15] Tammina, Srikanth. (2019). Transfer learning using VGG-16 with Deep Convolutional Neural Network for Classifying Images. International Journal of Scientific and Research Publications (IJSRP). 9. p9420. 10.29322/IJSRP.9.10.2019.p9420.

[16] Mukhometzianov, Rinat Carrillo, Juan. (2018). CapsNet comparative performance evaluation for image classification. 10.48550/arXiv.1805.11195.

[17] I. J. Goodfellow et al., "Multi-digit number recognition from street view imagery using deep convolutional neural networks," arXiv preprint arXiv:1312.6082, 2013.

[18] G. E. Hinton, "Shape representation in parallel systems," in *International Joint Conference on Artificial Intelligence*, vol. 2, 1981.

[19] G. E. Hinton, A. Krizhevsky, and S. D. Wang, "Transforming auto-encoders," in *International Conference on Artificial Neural Networks*, pp. 44–51, Springer, 2011.

[20] M. Jaderberg et al., "Spatial transformer networks," in *Advances in Neural Information Processing Systems*, pp. 2017–2025, 2015.

[21] Y. Netzer et al., "Reading digits in natural images with unsupervised feature learning," in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, vol. 2011, p. 5, 2011.

[22] B. A. Olshausen, C. H. Anderson, and D. C. Van Essen, "A neurobiological model of visual attention and invariant pattern recognition based on dynamic routing of information," *Journal of Neuroscience*, 13(11):4700–4719, 1993.

[23] L. Wan et al., "Regularization of neural networks using dropconnect," in *Proceedings of the 30th International Conference on Machine Learning (ICML-13)*, pp. 1058–1066, 2013.

[24] M. D. Zeiler and R. Fergus, "Stochastic pooling for regularization of deep convolutional neural networks," arXiv preprint arXiv:1301.3557, 2013.

Image Classification and Processing for Alzheimer's Disease Detection Using Deep Learning Algorithms

Aneesh Tufchi
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, Maharashtra-411043
Email: aneeshtufchi0@gmail.com

Shrihari Tiwari
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, Maharashtra-411043
Email: 951sht@gmail.com

Manshu Khajuria
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, Maharashtra-411043
Email: manshukhajuria01@gmail.com

Kshiteej Pawar
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, Maharashtra-411043
Email: kshiteejpawar2003@gmail.com

Prof. Virendra Bagade
Asst. Professor, Department of Computer Engineering
Pune Institute of Computer Technology
Pune, Maharashtra-411043
Email: vvbagade@pict.edu

Abstract

This study introduces a comparative analysis of deep learning models for clinical image classification, with a particular emphasis on brain MRI scans. Given the critical role of accurate and automated diagnosis in neurodegenerative diseases such as Alzheimer's, this study evaluates the effectiveness of Baseline neural networks like CNN, VGG16, ResNet, with Capsule Networks (CapsNet). The main goal is to analyze how CapsNet differs from traditional CNN-based architectures in handling spatial hierarchies and structural variations, and how its dynamic routing mechanism enhances classification performance in medical imaging. Each model exhibits unique advantages in clinical image analysis. CNNs are highly effective in capturing layered spatial characteristics, which makes them particularly suitable for recognizing patterns in brain tissues VGG16, with its deeper architecture and small 3×3 filters, enhances feature extraction, enabling the identification of subtle structural differences. ResNet mitigates the vanishing gradient problem through skip connections, allowing for improved learning in deep networks. CapsNet, unlike traditional CNN-based architectures, preserves spatial relationships between features, making it particularly promising for capturing positional variations in anatomical structures. The study evaluates these models based on key performance metrics, including classification accuracy, precision, recall, and F1-score. Preprocessing techniques such as image normalization, resizing, and data transformation (including rotation, flipping, and zooming) are employed to enhance model performance and mitigate overfitting, especially in limited data scenarios. By comparing baseline models with CapsNet, this study contributes to the growing research on deep learning in medical imaging, highlighting the advantages and limitations of different architectures in real-world healthcare applications.

Index Terms

CNN ,VGG ,ResNet, CapsNet ,Image Classification, Deep Learning, RoI, GANET , SBO ,ESBO

I. Introduction

Accurate and efficient image classification has become indispensable in modern medical diagnostics, particularly for the early detection and staging of diseases. Among various imaging techniques, brain MRI (Magnetic Resonance Imaging) is instrumental in diagnosing neurodegenerative disorders such as Alzheimer's disease, brain tumors, and multiple sclerosis. These conditions are frequently characterised by complex structural changes in the brain, making manual interpretation not just difficult but time-consuming. Detecting early-stage abnormalities can be particularly difficult because of their subtle nature. Consequently, the need for automated and highly precise image classification systems has grown, as they enhance diagnostic accuracy while simultaneously reducing the burden on radiologists.

Over the past few years, deep learning methods have seen widespread adoption in medical image analysis (Zhang Y, 2023)[7], with brain MRI scans being a prominent area of focus. Within the scope of machine learning, deep learning has revolutionized image recognition by enabling models to derive layered features directly from raw image data, eliminating the necessity for manual feature extraction. These models, when applied to medical imaging (Barrag´an-Montero A, 2021)[10], can uncover patterns that might not be immediately discernible to the human eye. This capability leads to improved accuracy in disease diagnosis, staging, and prognosis (Klang E, 2018)[6].

Convolutional Neural Networks (CNNs) are some of the most widely adopted deep learning networks for image classification, primarily due to their ability to capture spatial hierarchies in data (S Alawabi, 2017)[1]. They have been extensively applied to various medical imaging tasks, such as lesion detection, tumor segmentation, and brain disease classification (O'Shea, 2015)[2]. However, brain MRI scans pose unique challenges, including noise, anatomical variations, and limited dataset availability, necessitating more advanced CNN architectures for improved classification accuracy and generalization.

VGG (Visual Geometry Group) networks, a deep convolutional architecture, have demonstrated strong performance in image classification tasks. Their deep structure, combined with small kernel sizes (e.g., 3×3 filters), enables detailed feature extraction, allowing for the identification of subtle variations in medical images that may indicate early disease stages. The depth of VGG facilitates the detection of intricate patterns, enhancing its suitability for analyzing complex complex medical datasets where minor differences in image features can have significant diagnostic value.

Residual Networks (ResNet) build upon deep learning advancements by addressing the vanishing gradient problem that commonly arises in very deep networks. By incorporating residual connections, these models allow gradients to bypass certain layers, ensuring effective gradient propagation throughout the network. This capability is especially beneficial in medical image classification, where capturing high-level, intricate features across multiple layers is crucial for accurate disease staging. ResNet's ability to train deep networks without suffering performance degradation has positioned it as a highly successful architecture in medical imaging, particularly for distinguishing between early and advanced stages of conditions such as Alzheimer's.

Capsule Networks (CapsNet), introduced by Sabour and Hinton in 2017, were developed to overcome the limitations of conventional CNNs in recognizing spatial hierarchies and relationships between object components. Unlike CNNs, which often struggle with understanding how different parts of an object relate to each other, CapsNet explicitly models these spatial relationships, enhancing generalization and recognition accuracy (Haq, 2023)[8]. For example, while a CNN might misclassify an image of a disassembled bicycle simply based on the presence of its parts, a CapsNet can recognize that the wheels and handlebars are not correctly positioned relative to the frame and determine that the object is not a properly assembled bicycle. Essentially, CapsNet functions as a form of inverse graphics, representing objects through hierarchical relationships between their subcomponents. Its architecture consists of three key layers: the input layer, hidden layer, and output layer. Additionally, in terms of security, capsule networks have demonstrated greater resilience against certain adversarial attacks compared to traditional CNNs.

This research aims to analyze and compare the performance among three widely adopted deep learning architectures—CNN, VGG, ResNet with CapsNet—for classifying brain MRI images. The evaluation is based on key performance metrics, including accuracy, precision, recall, and F1-score, along with the models' ability to generalize across diverse datasets. Additionally, computational efficiency is taken into account, as real-time diagnostic applications require models that offer both high accuracy and speed and are also resource-efficient. The significance of this study lies in its potential to enhance automated medical diagnostics, particularly in the early detection and staging of neurodegenerative diseases. By conducting a comprehensive comparison of different deep learning models, this study seeks to identify their respective strengths and weaknesses, offering insights into the most suitable architectures for various medical image classification tasks. Moreover, the findings could contribute to the adoption of deep learning-based classification systems in clinical settings, ultimately facilitating more precise and timely diagnoses, thereby improving patient outcomes.

II. Literature Review

(S Alawabi, 2017)[1]Deep Learning refers to Artificial Neural Networks with multiple layers, known for their ability to process large volumes of data efficiently. Convolutional Neural Networks (CNNs), a type of deep learning model, excel in image and pattern recognition tasks. CNNs consist of various layers—convolutional, non-linearity, pooling, and fully connected—with only some having learnable parameters. This paper explores the components of CNNs, their working, and the factors affecting their performance.

(ALzubaidi,L et.al,2021)[3]Deep learning (DL) has become the leading approach in machine learning, outperforming traditional methods in various complex tasks across domains like NLP, cybersecurity, and medical imaging. This paper offers a comprehensive review of DL, covering its types, architectures—especially CNNs—and their evolution from AlexNet to HR.Net. It also discusses current challenges, applications, and the role of computational tools like GPUs and FPGAs. The goal is to provide a holistic understanding of DL and highlight research gaps.

(F. ALtaf, et.al 2019)[4]Deep learning is driving a major transformation in medical image analysis, gaining significant attention and even inspiring a dedicated conference in 2018. This paper critically reviews recent advancements, categorizing them by pattern recognition tasks and human anatomy. It aims to explain core deep learning concepts to non-experts and highlights the lack of annotated large-scale datasets as a key challenge. Drawing from computer vision and machine learning, it suggests promising solutions for advancing medical imaging.

(Li M, Jiang Y, et.al ,2023)[7]Deep learning (DL) has become crucial in medical image analysis, enabling real-time processing of complex data to improve healthcare outcomes. This review categorizes recent DL techniques—such as CNNs, RNNs, GANs, LSTMs, and hybrid models—based on key parameters like performance, methodology, and datasets. Python emerged as the dominant language, with most studies published in 2021. The paper highlights both advancements and ongoing challenges, offering insights to guide future research in medical image analysis.

(Haq, Mahmood S, et.al 2023)[8]Capsule Networks (CapsNets) are an advanced deep learning architecture introduced to preserve spatial and hierarchical relationships between features, unlike traditional CNNs. They use ”capsules” to capture vector outputs, encoding information like pose and orientation of image parts. This makes them more robust to adversarial attacks and better at generalizing with less data. CapsNets have shown strong performance in object recognition, security applications, and pattern analysis.

(Liang, Jiazhi. 2020)[12]Modern neural networks have grown significantly deeper, enabling them to learn complex features at various abstraction levels. However, training very deep networks introduces challenges like vanishing or exploding gradients, which slow down or destabilize learning. ResNet (Residual Network), introduced by Kaiming He's team, addressed these issues using residual connections that help gradients flow more effectively. This innovation allows for deeper, more accurate networks and has led to breakthroughs in image classification tasks.

(Tammina,S 2019)[15]Traditional machine learning and deep learning models struggle when applied to new domains with different data distributions, especially when labeled data is scarce. Transfer learning addresses this by leveraging knowledge from related tasks or domains to improve performance in the target domain. It enables the reuse of pre-trained models, making learning more efficient and effective for domains with limited data. This approach is particularly useful in scenarios like image classification, where common features can be transferred between tasks.This paper uses one of the pre-trained models VGG - 16 with Deep Convolutional Neural Network to classify images.

(G.E. Hinton ,et.al, 2017)[25]Capsule Networks (CapsNets) are groups of neurons whose vector outputs encode both the presence and properties (e.g., pose, orientation) of entities in an image. Unlike CNNs, CapsNets use dynamic routing-by-agreement to preserve spatial relationships and accurately assign parts to wholes. Each capsule predicts higher-level features, and strong agreement between predictions activates parent capsules. This method is more robust than max-pooling, handles overlapping objects better, and represents existence via vector length. The architecture scales positional precision through a shift from place-coding to rate-coding across layers.

(Patrick ML, et al., 2022) [27]Capsule Networks (CapsNets) are an advanced deep learning architecture proposed as an alternative to CNNs. Unlike CNNs, which use scalar values, CapsNets use vector outputs, allowing them to account for spatial relationships, pose, and deformations of objects. The capsules learn both the presence and properties of objects, enabling better recognition of whole entities by first recognizing their parts. CapsNet models can be divided into three types: transforming auto-encoders, vector capsules with dynamic routing, and matrix capsules with expectation-maximization routing. These networks are particularly effective in handling complex visual tasks, such as recognizing objects with varying poses.

III. Materials and Methods

A. Data Collection

The image classification system relies on a well-structured data collection process to obtain high-quality medical images, particularly brain MRI scans. These images are gathered from various repositories and databases, including:

Public datasets: Resources like the OASIS Dataset from Kaggle provide labeled MRI images of healthy individuals and patients at different stages of neurodegenerative diseases.

These datasets typically contain labeled images representing different stages of diseases like Alzheimer's, including Non-Demented, Very Mild Dementia, Mild Dementia, and Moderate Dementia. Labeled data is essential for training supervised learning models such as CNN, VGG, ResNet, and CapsNet.

B. Data PreProcessing

After collecting MRI images, they undergo a series of preprocessing steps to determine their compatibility with deep learning models. Effective preprocessing is essential, as the integrity of the input data significantly impacts model performance. The key steps include:

Resizing: MRI images vary in dimensions based on the imaging equipment used. To maintain consistency across the dataset,images are scaled to a standardized resolution (e.g., 256×256 or 128×128 pixels), ensuring uniform input for the models.

Normalization: Pixel intensity values in MRI scans can differ widely due to variations in scanning conditions. Normalizing these values to a fixed range, such as [0,1] or [-1,1], enhances numerical stability and accelerates neural network convergence.

Data Augmentation: Since clinical image datasets are often limited in size, augmentation techniques are employed to artificially expand the dataset and improve model generalization. Common augmentation strategies include:

Rotation: Applying random rotations to help the model recognize patterns regardless of orientation.

Flipping: Flipping images horizontally or vertically to introduce variability in training samples.

Zooming: Adjusting the scale of images to simulate differences in scan magnification.

Shifting: Slightly translating images to account for variations in patient positioning during MRI acquisition.

Image Slicing: MRI scans are three-dimensional, consisting of multiple two-dimensional slices captured at various depths. Depending on the model type, individual slices may be processed separately, or the entire 3D volume may be analyzed as a whole. For example, standard CNNs often classify individual slices, while 3D CNNs consider the full volume to retain spatial context.

By implementing these preprocessing techniques, the dataset is refined, standardized, and enhanced, allowing deep learning models to generalize more effectively and achieve robust performance on new data.

C. Model Building

1) CNN: A standard architecture for image classification, CNNs use convolutional layers to detect features and pooling layers to reduce spatial dimensions (Alzubaidi, 2021)[3]. CNNs are widely used in medical imaging because of their ability to handle large amounts of spatial data efficiently (F Altaf, 2019)[4]. The network learns to retrieve relevant information from MRI and classify them into disease stages.

- The convolutional layer applies a set of filters (or kernels) to the input image. For a given input image X, the output feature map Y after applying the filter W can be expressed as:

$$Y_{i,j,k} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} W_{m,n,k} \cdot X_{i+m,j+n,c} + b_k \tag{1}$$

  Where:
  - $Y_{i,j,k}$ is the output feature map at position (i, j) for the k-th filter.
  - $W_{m,n,k}$ is the weight of the k-th filter at position (m, n).
  - $X_{i+m,j+n,c}$ is the input value at position $(i + m, j + n)$ of the c-th channel.
  - b_k is the bias associated with the k-th filter.
  - M and N are the dimensions of the filter.

- Activation Function: After the convolution operation, an activation function (commonly ReLU) is applied element-wise:

$$Z_{i,j,k} = \text{ReLU}(Y_{i,j,k}) = \max(0, Y_{i,j,k}) \tag{2}$$

  Where:
  - $Z_{i,j,k}$ is the output after the activation function.

- Pooling Layer: Pooling layers reduce the spatial dimensions of the feature maps, typically using max pooling. The output after max pooling can be expressed as:

$$P_{i,j,k} = \max_{m,n} Z_{s \cdot i+m, s \cdot j+n,k} \tag{3}$$

  Where:
  - $P_{i,j,k}$ is the pooled output.
  - s is the stride used for pooling.

- m and n are the indices that iterate over the pooling window.

- Fully Connected Layer The output of the final pooling layer is flattened and fed into a fully connected layer. The output of this layer is calculated as:

$$O_j = \sum_{i=1}^{N} W_{ij} \cdot A_i + b_j \tag{4}$$

Where:
- O_j is the output of the j-th neuron in the fully connected layer.
- W_{ij} is the weight from neuron i to neuron j.
- A_i is the activation from the previous layer (flattened feature maps).
- b_j is the bias term for the j-th neuron.
- N is the number of inputs to the fully connected layer.

- Output Layer For classification tasks, the final output layer often uses the softmax function to convert the logits into probabilities:

$$P(y_k) = \frac{e^{O_k}}{\sum_{j=1}^{K} e^{O_j}} \tag{5}$$

Where:
- $P(y_k)$ is the probability of class k.
- O_k is the output of the k-th neuron in the output layer.
- K is the number of classes.

2) VGG: VGG extends CNNs by utilizing deeper networks, primarily with small 3×3 convolutional filters. It uses max pooling layers with a fixed stride of 2 and a 2x2 pool size, which helps to lower the spatial dimensions while maintaining the critical features. This increased depth allows the model to capture more complex and finer details in MRI images. VGG typically consists of multiple convolutional layers accompanied by max pooling operations and three dense layers at the end. The first two fully connected layers have 4096 neurons each, while the last corresponds to the number of classes in the classification task. It Primarily employs the ReLU (Rectified Linear Unit) activation function after each convolutional layer, which has become standard in many modern architectures.(T.Srinkanth, 2019)[15]

3) ResNet Residual Networks: It solves the problem of vanishing gradients in deep networks by introducing skip connections, which allow gradients to flow directly through the network without degradation(L,Jiazhi, 2020)[12]. ResNet is highly effective in medical image classification tasks, as it allows the model to learn intricate, deep representations of brain structures that distinguish between healthy and diseased brains.
The core building block of ResNet is the residual block, which combines the block's input with the output of the convolutional layers, facilitating more efficient training and enabling deeper architectures(K.He,X.Zhang,et.al , 2016) [13].

D. Baseline models vs CapsNet

Before exploring the architecture and functioning of capsule networks, it's important to first identify the distinctions between them and traditional CNN-based neural networks.

Hinton argued that accurately classifying and recognizing objects requires maintaining the hierarchical pose relationships between their parts. This approach is influenced by the way visual images are formed, using geometrical objects and matrices to represent relative positions and orientations. Known as inverse graphics, this concept reflects how the human brain interprets and recognizes objects. To mitigate the drawbacks of CNNs, Hinton(G.E. Hinton, et. al, 2019)[19] suggested replacing the scalar outputs of neurons — which merely signal the activation of replicated feature detectors — with vector-based outputs called capsules. Each capsule is responsible for identifying a specific visual component (such as a part of an object), and its vector output encodes both the likelihood of that component's presence and a set of 'instantiation parameters' describing its properties.
Capsule Networks (CapsNet) (S.Sabour, N.Forest, et.al,2017) [25] were introduced to address the limitations of traditional CNNs. Unlike CNNs, which typically rely on a backbone network for feature extraction, followed by fully connected layers and an N-way SoftMax function for classification, CapsNet enhances feature learning by preserving more detailed information

during feature aggregation. This allows the network to reason about object poses and part-whole relationships more effectively. CapsNet is composed of five key components: a non-shared transformation module, which applies unique transformation matrices to primary capsules to produce prediction vectors (votes); a dynamic routing layer, which groups input capsules into output capsules based on the consistency of their predictions; a squashing function that normalizes the length of capsule vectors to fall within the range [0,1); a marginal classification loss that works alongside the squashed outputs to perform classification; and a reconstruction loss that encourages the network to recreate the input image from the capsule outputs, ensuring the preservation of important features. CNNs often struggle to generalize well to unseen variations in input data (M.Alcorn , Q.Gong,et.al ,2019) [26], whereas CapsNet not only detects object components but also captures their properties, representing them as vectors. Lower-level capsules identify simpler patterns, while higher-level capsules assemble these into more complex structures through a hierarchical voting mechanism (T.Minh ,et.al,2022)[9].

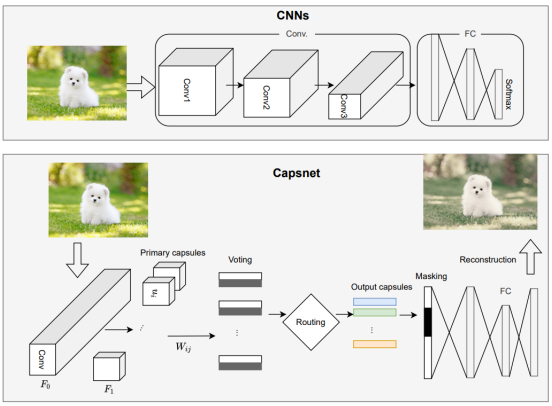


Fig. 1. Network Architechture comaprison between CNNs and CapsNet

We further assess the robustness of CapsNet in comparison to CNNs across the following aspects:

- Translation Invariance: CNNs can recognize whether an object exists in a specific area, but they struggle to capture the spatial relationships among multiple objects. This shortcoming limits their potential to model spatial dependencies among different features. As demonstrated in Fig. 2, while a CNN can classify an image as containing a dog or a human face, it cannot determine the relative position of the dog with respect to the background or the arrangement of facial features.
- Reduced Data Requirement for Generalization: CNNs require large datasets featuring objects in a variety of positions and orientations to learn and generalize effectively. These models depend on extensive training using diverse samples, under the assumption that encountering enough variations will improve their ability to recognize unseen instances. To aid this process, CNNs often apply data augmentation, creating multiple altered versions of each sample. On the other hand, CapsNet inherently captures spatial relationships among an object's components through its learned weights, enabling it to identify objects across varying positions and orientations, while minimizing reliance on extensive data augmentation. By learning invariant part-whole relationships, CapsNet demonstrates better generalization to new, unseen variations.
- Improved Interpretability: Despite the development of various techniques to explain CNN predictions, model interpretability remains a significant challenge. CapsNet addresses this by integrating part-whole relationships directly into its architecture, improving the transparency and explainability of its predictions. Higher-level capsules in CapsNet often capture meaningful, semantic concepts, making the model more interpretable compared to conventional neural networks. The disentangled representations learned by CapsNet frequently align with human-understandable visual attributes, such as object rotation, position, and scale.

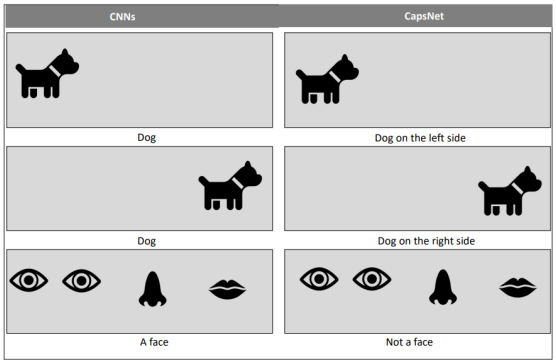


Fig. 2. Translation invariant comparison between CNNs and CapsNet

IV. CAPSNET

Capsule Networks, along with the concept of dynamic routing between capsules, were introduced by Hinton and his collaborators in 2017 through their research paper (S.Sabour, N.Forest, et.al,2017) [25]. This architecture demonstrated notable performance improvements on the MNIST digit dataset, outperforming Convolutional Neural Networks (CNNs) while using a relatively small number of trainable parameters(S.Sabour, N.Forest, et.al,2017)(Patrick M.K, et.al ,2022)(J.Rajasegaran ,et.al, 2019) [25,27,28]. One of the key reasons for this advantage lies in the limitations of CNNs, particularly their use of pooling layers for down-sampling feature maps. This process leads to the loss of fine-grained details in images, preventing CNNs from effectively capturing subtle spatial information. Capsule Networks were specifically designed to address this issue by preserving spatial hierarchies and fine details. Unlike traditional neurons that output scalars, capsules produce vectors capable of representing both the presence of a feature and its various properties, such as orientation and position, thereby enabling the network to distinguish images based on these multidimensional characteristics.

A. Architechture of Capsule Network

The Capsule Network is applied for classifying brain MRI images from the OASIS dataset. A typical CapsNet consists of six layers, serving both encoding and decoding functions. The architecture starts with a convolutional layer and proceeds to a primary capsule layer, which applies an additional convolutional operation along with a squashing function. It then proceeds to a digit capsule (or label capsule) layer, where dynamic routing between capsules takes place. The final three layers are fully connected (FC) layers responsible for reconstructing the input image, as illustrated in Fig. 3.

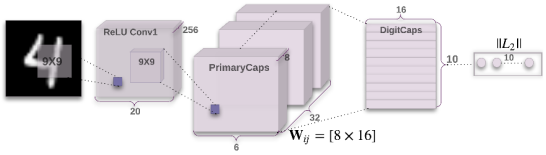


Fig. 3. Capsnet encoder architechture

Unlike convolutional layers that use various activation functions to activate neurons, the primary capsule layer applies a non-linear squashing function(Mukhometzianov, Rinat Carrillo, Juan,2018) [16], as shown in (6), to transform the neuron outputs into vector form.

$$v_j = \frac{\|S_j\|^2}{1 + \|S_j\|^2} \frac{S_j}{\|S_j\|} \tag{6}$$

Here, v_j represents the output vector of capsule j, while S_j denotes the total input to that capsule. As shown in (8), the total input S_j is calculated as a weighted sum of all the prediction vectors $\hat{P}^{j|i}$ coming from the capsules in the lower layer. Each prediction vector $\hat{P}^{j|i}$ is obtained using (7), where it is computed by multiplying the output p_i of the lower-layer capsule by the corresponding weight matrix W_{ij}.

$$\hat{P}_{j|i} = W_{ij} p_i \tag{7}$$

$$S_j = \sum_i k_{ij} \hat{P}_{j|i} \tag{8}$$

Here, k_{ij} is the coupling coefficient, computed using (9) through the dynamic routing algorithm(S.Sabour, N.Forest, et.al,2017) [25]. Dynamic routing is a mechanism that determines how information flows between capsules within the network. It works based on a process called routing-by-agreement, where higher-level capsules iteratively receive input from lower-level capsules based on the level of agreement between their predictions. This iterative refinement allows the network to dynamically adjust connections, improving the accuracy of part-whole relationships. It is generally recommended to limit the number of routing iterations to three, as increasing the number of iterations may lead to overfitting.

$$k_{ij} = \frac{\exp(n_{ij})}{\sum_k \exp(n_{ik})} \tag{9}$$

Here, n_{ij} represents the log probability, which is computed using the softmax function. The sum of the coupling coefficients k_{ij} between capsule i and the top layer must equal 1. To compute k_{ij}, the log probability n_{ij} is first calculated using (10).

$$n_{ij} \leftarrow n_{ij} + \hat{P}_{j|i} \cdot v_j \tag{10}$$

Initially, the value of n_{ij} is set to 0. From this, we can compute k_{ij} and p_i, where p_i is the output from the previous layer of capsules. Using these three values, the input to the next-level capsule, S_j, can be calculated. For the margin loss, it is computed for each class, as shown in (11), to determine whether a particular object belongs to that class.

$$M_l = C_k \max(0, u^+ - \|v_k\|)^2 + \lambda(1 - C_k)\max(0, \|v_k\| - u^-)^2 \tag{11}$$

Here, $C_k = 1$ if a digit of class k is present, and $u^+ = 0.9$ and $u^- = 0.1$ are hyperparameters that control the down-weighting of the loss. The total loss is the sum of the losses from all the digit capsules (Mazzia, v. ,et.al,2021)[11].

B. Dynamic Routing Between the Capsules

The network comprises of three primary components: the Convolutional layer, the Primary Capsule (PC) layer, and the Class Capsule layer. The Primary Capsule layer is the first capsule-based layer, followed by a series of intermediate capsule layers, and concludes with the Class Capsule layer.

Feature extraction is carried out by the convolutional layer, and its output is forwarded to the Primary Capsule layer. Each capsule i (where $1 \leq i \leq N$) in layer l has an activity vector $u_i \in \mathbb{R}$, which encodes spatial information through instantiation parameters.

The output vector u_i from the i-th lower-level capsule is passed to all capsules in the subsequent layer $l + 1$. The j-th capsule at layer $l + 1$ receives u_i and computes the product with the corresponding weight matrix W_{ij}. The resulting vector $\hat{P}_{j|i}$ represents the prediction made by capsule i for the entity encoded by capsule j in layer $l + 1$.

$$\hat{P}_{j|i} = W_{ij} p_i \tag{12}$$

Each prediction vector is multiplied by a coupling coefficient c_{ij}, which measures the level of agreement between capsule i and capsule j. A higher degree of agreement increases the coupling coefficient, while a lower agreement reduces it. The total input s_j to capsule j is obtained using a weighted summation of all prediction vectors sent to it.

$$s_j = \sum_{i=1}^{N} c_{ij} \hat{P}_{j|i} \tag{13}$$

This input is propagated through a non-linear *squashing function* to produce the final output vector v_j:

$$v_j = \frac{\|s_j\|^2}{1 + \|s_j\|^2} \cdot \frac{s_j}{\|s_j\|} \tag{14}$$

The coupling coefficients c_{ij} are calculated by a softmax function applied over logits b_{ij}:

$$c_{ij} = \frac{\exp(b_{ij})}{\sum_k \exp(b_{ik})} \tag{15}$$

These coefficients are dynamically updated during training to reflect the degree of agreement (i.e., routing-by-agreement) between capsules across layers.

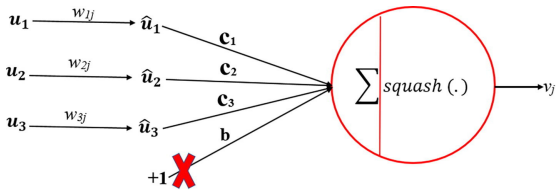


Fig. 4. Representation of a Capsule

The squashing function ensures that the output length from each capsule remains between 0 and 1, similar to a likelihood. The vector v_j from one capsule layer is passed to the next layer's capsules and processed in the same manner as previously described. The coupling coefficient c_{ij} ensures that the prediction from capsule i at layer l is connected to the prediction from capsule j at layer $l+1$.

In each iteration, c_{ij} is updated by calculating the dot product between $\hat{P}_{j|i}$ and v_j. Specifically, the vector values associated with each capsule can be viewed as a pair of numbers the probability, which represents the likelihood of the feature encapsulated by the capsule, and a set of instantiation parameters that explain the consistency between the layers.

Thus, the relevance of the path by agreement arises from the fact that when lower-level capsules agree with a higher-level capsule, they "construct a part-whole" relationship, indicating the relevance of the path. This process is known as dynamic routing-by-agreement, as demonstrated in Algorithm 1.

Algorithm 1 Dynamic Routing Algorithm

procedure ROUTING($\hat{P}_{j|i}, r, l$)
  for all capsule i in layer l and capsule j in layer $l+1$ **do**
    $b_{ij} \leftarrow 0$
  end for
  for r iterations **do**
    for all capsule i in layer l **do**
      $c_i \leftarrow \text{softmax}(b_i)$ ▷ Softmax computes c_{ij}
    end for
    for all capsule j in layer $l+1$ **do**
      $s_j \leftarrow \sum_i c_{ij} \hat{P}_{j|i}$
    end for
    for all capsule j in layer $l+1$ **do**
      $v_j \leftarrow \text{squash}(s_j)$ ▷ Squash computes v_j
    end for
    for all capsule i in layer l and capsule j in layer $l+1$ **do**
      $b_{ij} \leftarrow b_{ij} + \hat{P}_{j|i} \cdot v_j$
    end for
  end for
  return v_j
end procedure

C. CapsNet with U-Net

Capsule Networks are designed to overcome key CNN limitations, especially the loss of spatial relationships due to pooling layers. CNNs (like those used in U-Net) extract hierarchical features but fail to preserve pose and orientation information — crucial in medical imaging where structural accuracy is vital.

Hence, integrating Capsule layers into U-Net enhances its feature representation capacity by:

Preserving the spatial hierarchy.

Making the model robust to rotations, deformations, and scale variations.

Enabling richer and more interpretable embeddings.

1) U-Net: U-Net is a convolutional neural network with:

Encoder: Series of convolution + pooling layers that capture contextual features.

Decoder: Series of upsampling + convolution layers that recover spatial resolution.

Skip connections: Allow transfer of low-level features directly from encoder to decoder, preserving fine-grained details.

2) Where CapsNet fits in U-Net: The most effective integration point in U-Net is the bottleneck or deep encoder layer.
Architecture Outline: Encoder:
Standard convolution + ReLU + pooling layers.
Deeper features are passed to Capsule Layer.
Capsule Bottleneck:
Input features reshaped into primary capsules.
Dynamic Routing applied to form higher-level capsules (like class capsules in the paper).
Vectorized output captures more semantically rich features with spatial relationships.
Decoder:
Deconvolution (upsampling) layers reconstruct the image.
Skip connections use both raw features and capsule-enhanced embeddings for precise reconstruction.

V. Results

Measuring Type 1 and Type 2 errors through these metrics allows you to better understand and evaluate the performance of your machine learning models. Depending on the specific use case, different metrics may be prioritized to achieve the desired balance of accuracy and error types In the context of machine learning and statistical classification, Type 1 and Type 2 errors can be measured using specific metrics derived from the confusion matrix. Following the metrics related to these errors:

Type 1 Error (False Positive)

Definition

A Type 1 error occurs when the model incorrectly identifies a negative instance as positive. This means that the model predicts the presence of a condition (e.g., a disease) when it is not actually present.

Metrics Associated

1) Precision:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Precision indicates the quality of positive predictions. It answers the question: "Of all instances classified as positive, how many were actually positive?" A high precision means fewer false positives.

Type 2 Error (False Negative)

Definition

A Type 2 error occurs when the model fails to identify a positive instance, incorrectly classifying it as negative. This means that the model predicts the absence of a condition when it is actually present.

Metrics Associated

2) Recall (Sensitivity or True Positive Rate):

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Recall measures the model's ability to identify all relevant instances. It answers the question: "Of all actual positives, how many were correctly identified?" A high recall means fewer false negatives.

General Metrics (Not Categorized as Type 1 or Type 2)

3) Accuracy:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Accuracy measures the overall correctness of the model across all instances. It answers the question: "What proportion of total instances were correctly classified?" While useful, accuracy can be misleading, especially in imbalanced datasets where one class significantly outweighs the other.

TABLE I
COMPARISON OF PERFORMANCE METRICS FOR DIFFERENT MODELS

Measure	CNN	VGG16	ResNet	CapsNet	CapsNet+UNet
Accuracy	0.76	0.73	0.77	0.83	0.77
Precision	0.71	0.71	0.75	0.76	0.60
Recall	0.75	0.75	0.78	0.83	0.77
F1 Score	0.73	0.73	0.68	0.79	0.68

4) F1 Score:

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1 score combines precision and recall into a single metric, providing a balance between the two. It's particularly useful when you need to account for both false positives and false negatives. A high F1 score indicates a model that has both good precision and recall.

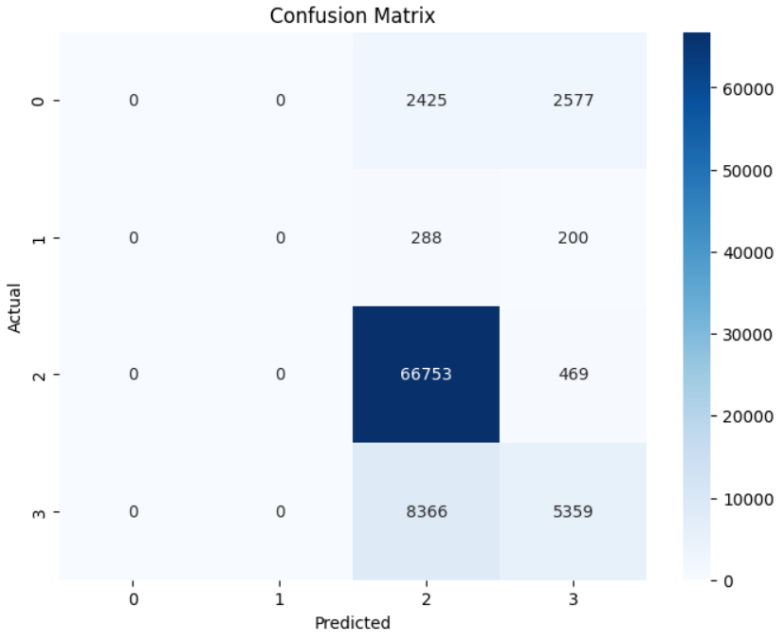


Fig. 5. CapsNet

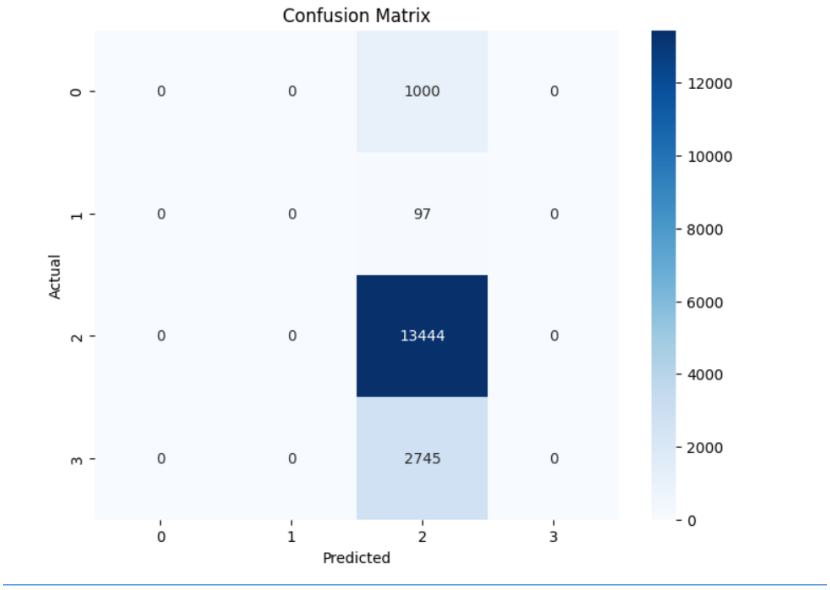


Fig. 6. CapsNet+UNet

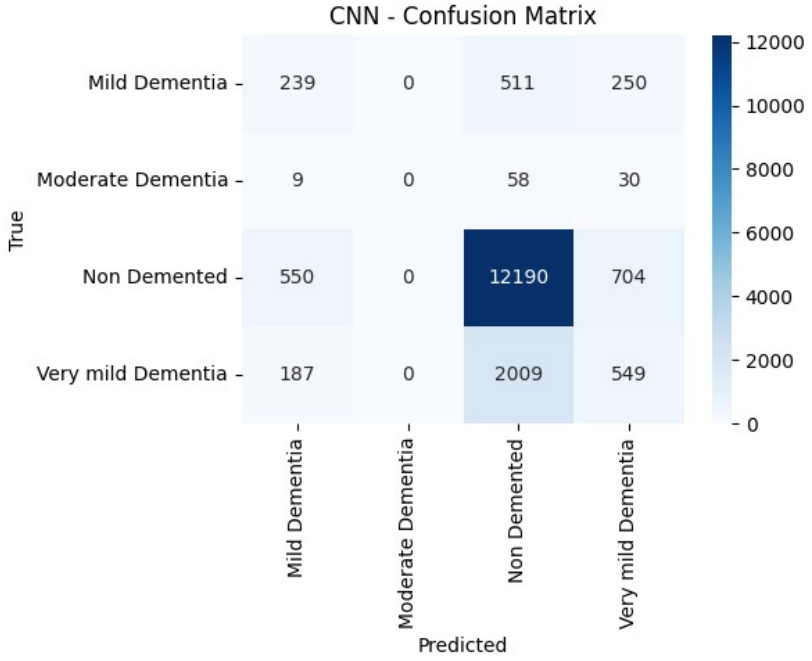


Fig. 7. CNN

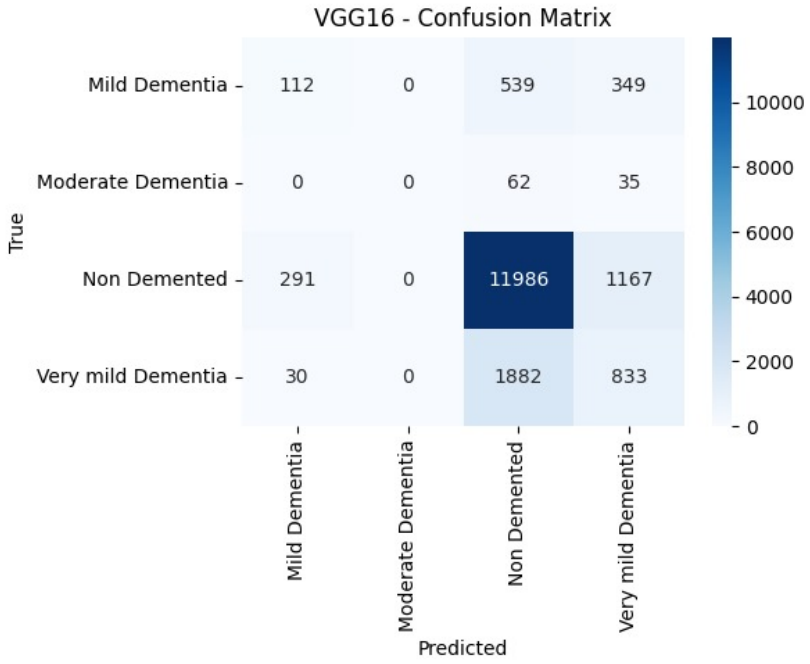


Fig. 8. VGG16

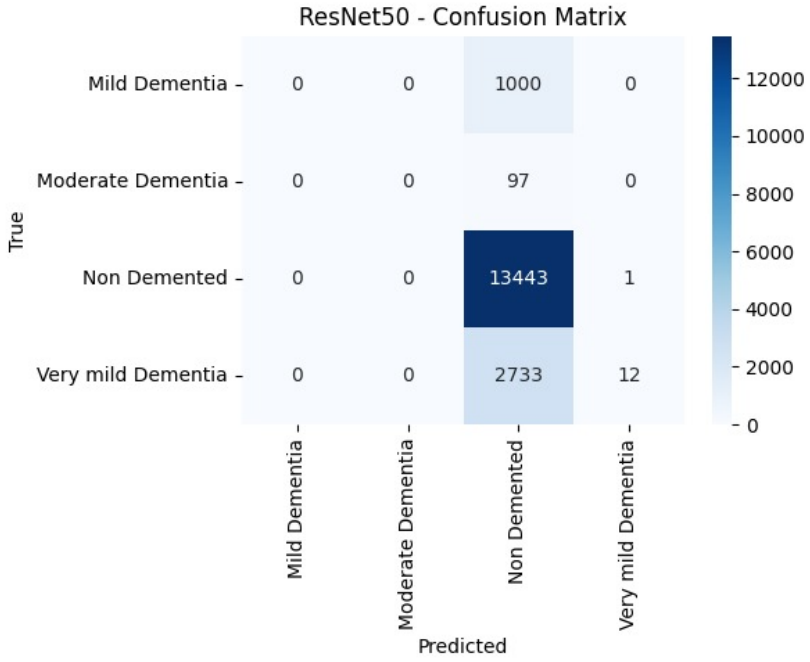


Fig. 9. ResNet

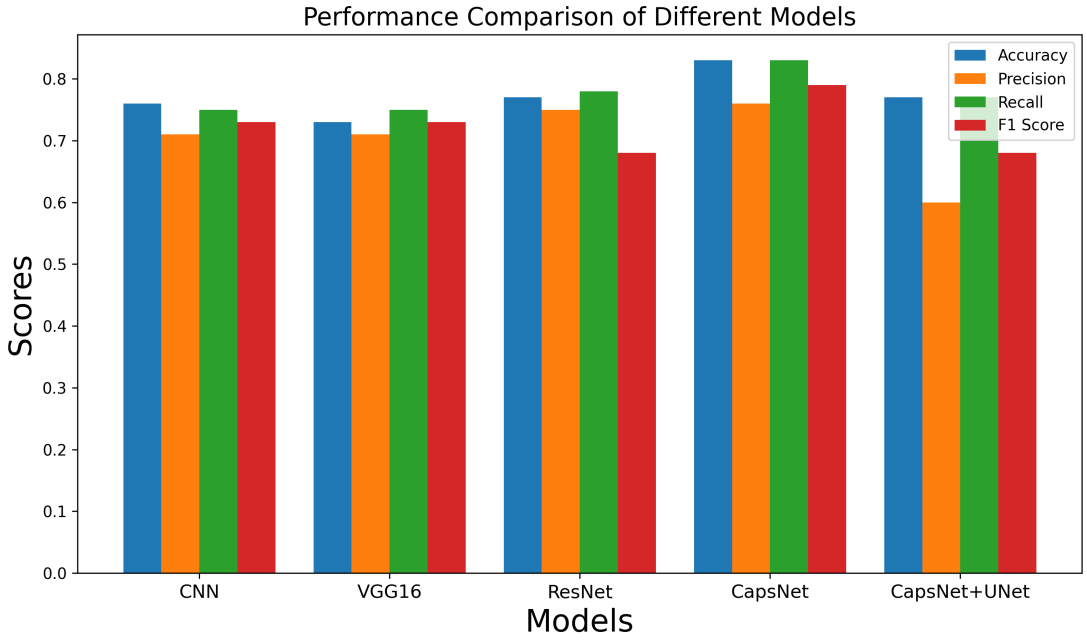


Fig. 10. Performance Comparison of different models

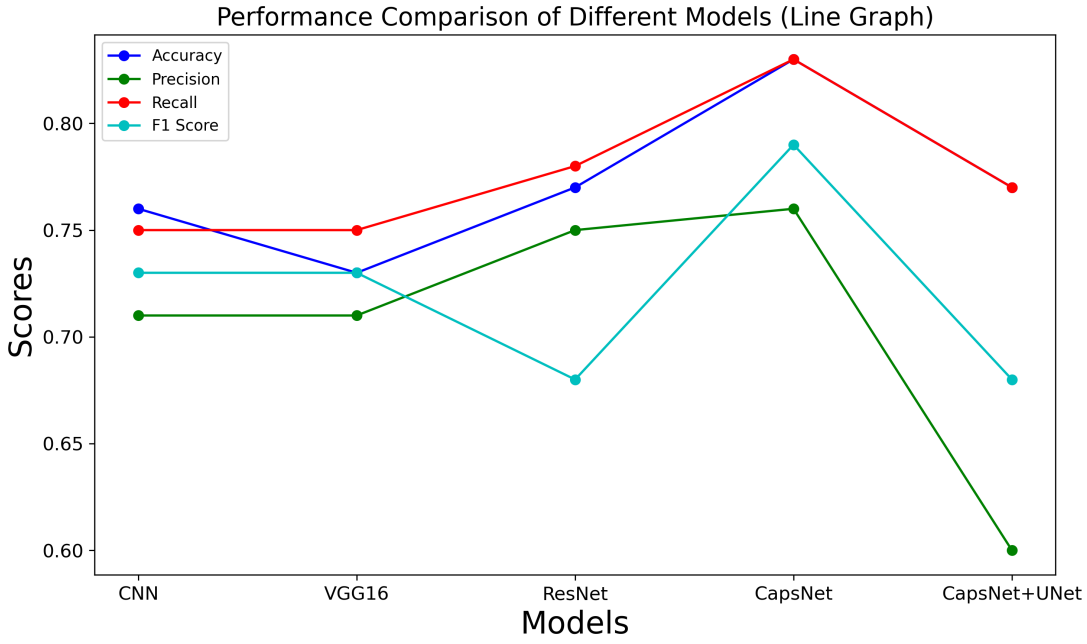


Fig. 11. Performance Comparison of different models (Line Graph)

VI. Conclusion

This study underscores the pivotal role of deep learning in advancing medical image classification, particularly in the context of brain MRI analysis for diagnosing neurodegenerative diseases like Alzheimer's. Through a comprehensive comparison of CNN, VGG16, ResNet, and CapsNet architectures, the research highlights the unique strengths and limitations of each model. While CNNs and VGG16 excel in feature extraction and pattern recognition, and ResNet addresses challenges related to deep network training, CapsNet is recognized for being able to preserve spatial hierarchies and relationships—an essential aspect in medical imaging. The inclusion of CapsNet demonstrates its potential to improve classification accuracy by effectively handling positional variations in anatomical structures. Furthermore, the application of image preprocessing techniques such

as normalization, augmentation, and resizing enhances model generalization, particularly when data is limited. By evaluating performance across metrics like accuracy, precision, recall, and F1-score, This work presents important findings regarding model suitability for real-world clinical implementation. Ultimately, the findings support the integration of robust deep learning systems into medical diagnostics, promising improved accuracy, efficiency, and early disease detection.

REFERENCES

[1] S. Albawi, T. A. Mohammed and S. Al-Zawi, "Understanding of a convolutional neural network," 2017 International Conference on Engineering and Technology (ICET), Antalya, Turkey, 2017, pp. 1-6, doi: 10.1109/ICEngTechnol.2017.8308186.

[2] O'Shea, Keiron Nash, Ryan. (2015). An Introduction to Convolutional Neural Networks. ArXiv e-prints.

[3] Alzubaidi, L., Zhang, J., Humaidi, A.J. et al. Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. J Big Data 8, 53 (2021). https://doi.org/10.1186/s40537-021-00444-8

[4] F. Altaf, S. M. S. Islam, N. Akhtar and N. K. Janjua, "Going Deep in Medical Image Analysis: Concepts, Methods, Challenges, and Future Directions," in IEEE Access, vol. 7, pp. 99540-99572, 2019, doi: 10.1109/ACCESS.2019.2929365.

[5] Mittal, Ansh Kumar, Deepika. (2019). AiCNNs (Artificially-integrated Convolutional Neural Networks) for Brain Tumor Prediction. EAI Endorsed Transactions on Pervasive Health and Technology. 5. 10.4108/eai.12-2-2019.161976.

[6] Klang E. Deep learning and medical imaging. J Thorac Dis. 2018 Mar;10(3):1325-1328. doi: 10.21037/jtd.2018.02.76. PMID: 29708147; PMCID: PMC5906243.

[7] Li M, Jiang Y, Zhang Y, Zhu H. Medical image analysis using deep learning algorithms. Front Public Health. 2023 Nov 7;11:1273253. doi: 10.3389/fpubh.2023.1273253. PMID: 38026291; PMCID: PMC10662291.

[8] Haq, Mahmood Ul Sethi, Muhammad Rehman, Atiq. (2023). Capsule Network with Its Limitation, Modification, and Applications—A Survey. Machine Learning and Knowledge Extraction. 5. 891-921. 10.3390/make5030047.

[9] Tran, Minh Vo, Khoa Quinn, Kyle Nguyen, Hien Luu, Khoa Le, Ngan. (2022). CapsNet for Medical Image Segmentation. 10.48550/arXiv.2203.08948.

[10] Barragán-Montero A, Javaid U, Valdés G, Nguyen D, Desbordes P, Macq B, Willems S, Vandewinckele L, Holmström M, Löfman F, Michiels S, Souris K, Sterpin E, Lee JA. Artificial intelligence and machine learning for medical imaging: A technology review. Phys Med. 2021 Mar;83:242-256. doi: 10.1016/j.ejmp.2021.04.016. Epub 2021 May 9. PMID: 33979715; PMCID: PMC8184621.

[11] Mazzia, V., Salvetti, F. Chiaberge, M. Efficient-CapsNet: capsule network with self-attention routing. Sci Rep 11, 14634 (2021). https://doi.org/10.1038/s41598-021-93977-0

[12] Liang, Jiazhi. (2020). Image classification based on RESNET. Journal of Physics: Conference Series. 1634. 012110. 10.1088/1742-6596/1634/1/012110.

[13] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 770-778, doi: 10.1109/CVPR.2016.90.

[14] Simonyan, Karen Zisserman, Andrew. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv 1409.1556.

[15] Tammina, Srikanth. (2019). Transfer learning using VGG-16 with Deep Convolutional Neural Network for Classifying Images. International Journal of Scientific and Research Publications (IJSRP). 9. p9420. 10.29322/IJSRP.9.10.2019.p9420.

[16] Mukhometzianov, Rinat Carrillo, Juan. (2018). CapsNet comparative performance evaluation for image classification. 10.48550/arXiv.1805.11195.

[17] I. J. Goodfellow et al., "Multi-digit number recognition from street view imagery using deep convolutional neural networks," arXiv preprint arXiv:1312.6082, 2013.

[18] G. E. Hinton, "Shape representation in parallel systems," in *International Joint Conference on Artificial Intelligence*, vol. 2, 1981.

[19] G. E. Hinton, A. Krizhevsky, and S. D. Wang, "Transforming auto-encoders," in *International Conference on Artificial Neural Networks*, pp. 44–51, Springer, 2011.

[20] M. Jaderberg et al., "Spatial transformer networks," in *Advances in Neural Information Processing Systems*, pp. 2017–2025, 2015.

[21] Y. Netzer et al., "Reading digits in natural images with unsupervised feature learning," in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, vol. 2011, p. 5, 2011.

[22] B. A. Olshausen, C. H. Anderson, and D. C. Van Essen, "A neurobiological model of visual attention and invariant pattern recognition based on dynamic routing of information," *Journal of Neuroscience*, 13(11):4700–4719, 1993.

[23] L. Wan et al., "Regularization of neural networks using dropconnect," in *Proceedings of the 30th International Conference on Machine Learning (ICML-13)*, pp. 1058–1066, 2013.

[24] M. D. Zeiler and R. Fergus, "Stochastic pooling for regularization of deep convolutional neural networks," arXiv preprint arXiv:1301.3557, 2013.

[25] Sara Sabour, Nicholas Frosst, and Geoffrey E. Hinton. Dynamic routing between capsules. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 3856–3866, 2017.

[26] Michael A. Alcorn, Qi Li, Zhitao Gong, Chengfei Wang, Long Mai, Wei-Shinn Ku, and Anh Nguyen. Strike (with) a pose: Neural networks are easily fooled by strange poses of familiar objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4845–4854, 2019.

[27] Patrick M. K., Adekoya A. F., Mighty A. A., and Edward B. Y. Capsule networks – a survey. *Journal of King Saud University - Computer and Information Sciences*, 34(1):1295–1310, 2022.

[28] J. Rajasegaran, V. Jayasundara, S. Jayasekara, H. Jayasekara, S. Seneviratne, and R. Rodrigo. DeepCaps: Going deeper with capsule networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10725–10733, 2019.